Bengaluru North University

Project Report on

AUTOMATIC INDIAN SIGN LANGUAGE TRANSLATOR

Submitted in partial fulfillment of the requirement for the award of the degree in

Master of Computer Applications

ACADEMIC YEAR – 2024-25

Submitted By

ASHWIN KUMAR A
P19UU23S126017

Project Guide

Mrs. PRASANTHI K
Assistant Professor
Department of Computer Science and Applications

S.E.A. College of Science, Commerce and Arts

Affiliated to Bengaluru North University
Recognized by UGC under 2(f), Accredited by NAAC at 'B++' Grade
K.R Puram, Bangalore – 560 049

College Code : 19UU
AISHE Code C-21052

ಎಸ್.ಈ.ಎ ವಿಜ್ಞಾನ, ವಾಣಿಜ್ಯ ಮತ್ತು ಕಲಾ ಕಾಲೇಜು

(ಬೆಂಗಳೂರು ಉತ್ತರ ವಿಶ್ವವಿದ್ಯಾಲಯದಿಂದ ಸಂಯೋಜನೆಗೊಂಡಿದೆ ಹಾಗೂ ಕರ್ನಾಟಕ ಸರ್ಕಾರದಿಂದ ಮಾನ್ಯತೆ ಪಡೆದಿದೆ)

S.E.A COLLEGE OF SCIENCE, COMMERCE AND ARTS

(Affiliated to Bengaluru North University, and Recognized by Govt.of Karnataka)

NAAC Accredited with 'B++' Grade

Ektanagar, A. Krishnappa Circle, Ayyappanagar, Devasandra Main Road, Virgonagar Post, K.R. Puram, Bengaluru - 560 049.

Tel. : 25613741 / 42 Fax : 25613418 Mob : 9900732511 E-Mail:priseadegree@gmail.com Website : www.seadegree.ac.in

CERTIFICATE

This is to certify that the dissertation titled **AUTOMATIC INDIAN SIGN LANGUAGE TRANSLATOR** is an original work of **Ashwin Kumar A** bearing University Register Number **P19UU23S126017** is being submitted in partial fulfilment for the award of the Master's Degree in Computer Application of Bengaluru North University. The report has not been submitted earlier either to this University / Institution for the fulfilment of the requirement of a course of study.

Signature of the Guide
Mrs. Prasanthi K
Assistant Professor
Department Computer Science and Applications

Signature of the HOD
Mr. Manjunatha S
Head of the Department
Computer Science and Applications

Signature of the Principal
Dr. Muthe Gowda T N,
Principal

Valued By

Examiners

1.

2.

College Code : 19UU
AISHE Code C-21052

ಎಸ್.ಈ.ಎ ವಿಜ್ಞಾನ, ವಾಣಿಜ್ಯ ಮತ್ತು ಕಲಾ ಕಾಲೇಜು
(ಬೆಂಗಳೂರು ಉತ್ತರ ವಿಶ್ವವಿದ್ಯಾಲಯದಿಂದ ಸಂಯೋಜನೆಗೊಂಡಿದೆ ಹಾಗೂ ಕರ್ನಾಟಕ ಸರ್ಕಾರದಿಂದ ಮಾನ್ಯತೆ ಪಡೆದಿದೆ)
S.E.A COLLEGE OF SCIENCE, COMMERCE AND ARTS
(Affiliated to Bengaluru North University, and Recognized by Govt.of Karnataka)
NAAC Accredited with 'B++' Grade
Ektanagar, A. Krishnappa Circle, Ayyappanagar, Devasandra Main Road, Virgonagar Post, K.R. Puram, Bengaluru - 560 049.
Tel. : 25613741 / 42 Fax : 25613418 Mob : 9900732511 E-Mail:priseadegree@gmail.com Website : www.seadegree.ac.in

GUIDE CERTIFICATE

This is to certify that the content of this project entitled, **AUTOMATIC INDIAN SIGN LANGUAGE TRANSLATOR** by **Ashwin Kumar A, P19UU23S126017** is a bona fide work, submitted to Bengaluru North University, for consideration in partial fulfilment of the requirement of the Master's Degree in Computer Application.

The original research work was carried out by students under my supervision in the academic year 2024-25. On the basis of the declaration made by Students, I recommend this project report for evaluation.

CERTIFIED BY
Mrs. Prasanthi K
Assistant Professor
Department of Computer
Science and Applications

DECLARATION

We hereby declare that **AUTOMATIC INDIAN SIGN LANGUAGE TRANSLATOR** is the result of the project work carried out by us under the guidance of **Mrs. Prasanthi K** in partial fulfilment for the award of Master's Degree in Computer Application by Bangalore North University.

We also declare that this project is the outcome of our own efforts and that it has not been submitted to any other university or Institute for the award of any other degree or Diploma or Certificate.

(SIGNATURE)

Place: Bangalore
Date:

Ashwin Kumar A
P19UU23S126017

ACKNOWLEDGMENT

Above all, we want to thank God Almighty for the blessings, health, wisdom, knowledge, and strength he has given to us. It is only by his will that we are able to start and complete this project.

We wish to express our sincere gratitude to the **Late A. Krishnappa**, the founder of South East Asian Educational Trust for providing good infrastructure and adequate equipment's for the proper functioning of the institution.

We also thank **Dr. Muthe Gowda T.N,** the Principal of S.E.A. College of Science, Commerce and Arts for his tireless effort in managing the College making sure that everything runs in proper order.

We sincerely thank **Mr. Manjunatha S,** the Head of the Department of Computer Science and Applications for his constant support, advice, and guidance which greatly contributed to the successful completion of this project. We also wish to express our gratitude to the all the staff in Department of Computer Science and Applications of their support, concern and encouragement not only to the project but also in our studies.

We sincerely thank **Mrs. Prasanthi K** Assistant Professor in Department of Computer Science and Applications for her constant support, advice, and guidance which greatly contributed to the successful completion of this project.

We also want to thank our parents for their guidance, support, encouragement, advice and prayers. And also, for providing all the materials we need, not only for our studies but also for our well-being if not for them, we will not be here today.

ABSTRACT

Communication is one of the most fundamental aspects of human interaction, yet individuals with hearing or speech impairments often face significant challenges in expressing themselves to those unfamiliar with sign language. The Sign Language Translator project aims to address this communication gap through an intelligent, real-time gesture recognition and translation system. The primary objective of the system is to detect and interpret hand gestures representing sign language alphabets and words, and translate them into readable text or audible speech.

The proposed system integrates computer vision, machine learning, and deep learning techniques to achieve accurate gesture recognition. It utilizes TensorFlow as the core deep learning framework, supported by a custom-trained convolutional neural network (CNN) model. The model is trained using a large dataset of gesture images representing the Indian Sign Language (ISL) alphabet and commonly used phrases. The trained model (hand.h5) enables high-precision classification of real-time hand gestures captured through a webcam or other image input sources.

The front-end interface is developed using Python and Django, providing a user-friendly platform for interaction. The Django framework handles routing, input processing, and real-time communication between the deep learning model and the user interface. The detect_gesture.py module is responsible for gesture detection, preprocessing video frames, and feeding them into the trained model for prediction. The recognized gestures are then converted into corresponding textual output, which is optionally transformed into speech using a text-to-speech engine. This multi-modal output design ensures accessibility for both hearing and non-hearing users.

The system's modular design allows for scalability, enabling future enhancements such as bidirectional translation, multilingual support, and integration with mobile or IoT platforms. Through this project, the potential of artificial intelligence in improving human communication and inclusivity is demonstrated. The Sign Language Translator contributes meaningfully to assistive technology by empowering individuals with hearing impairments to communicate more freely and effectively with the broader community.

Index

1. INTRODUCTION

1.1 Introduction

Communication is one of the most essential aspects of human life. It enables individuals to share ideas, thoughts, and emotions effectively. For most people, verbal communication using spoken language is natural and effortless. However, for individuals with hearing or speech impairments, communication with those who do not understand sign language remains a significant challenge. Sign language is a structured form of communication that uses specific hand gestures, facial expressions, and body movements to represent words and phrases. Although it is a powerful and expressive medium, its limited understanding among the general population often results in communication barriers.

To overcome this limitation, technology can play a vital role by bridging the gap between the hearing and non-hearing communities. The *Sign Language Translator* project was developed with the objective of facilitating smoother communication by translating Indian Sign Language (ISL) into text and speech, and vice versa. Instead of relying on artificial intelligence or complex models, this project uses a rule-based mapping system and a predefined database of images, GIFs, and words. Each sign or gesture corresponds to a specific word, phrase, or alphabet stored in the system. The translator retrieves and displays these representations dynamically based on user input.

The project has been implemented using the Python programming language and the Django web framework, which together provide a robust, modular, and interactive platform. The interface allows users to input text, select signs, and view their corresponding visual or audio translations. The integration of text-to-speech conversion enhances accessibility by allowing the translated text to be spoken aloud. Similarly, for sign learning and communication, the system displays GIFs and images representing Indian Sign Language alphabets and common phrases. This approach makes it useful not only as a communication tool but also as an educational aid for individuals learning sign language.

The system’s design is simple, efficient, and user-friendly. It operates without requiring any external servers or internet connectivity for translation, as all resources are stored locally within the project database. This ensures fast response times and reliable performance even in offline environments. The modular structure of the Django framework makes the system easy to maintain and expand. For instance, additional signs, phrases, or new features like multi-language support can be integrated into the existing structure with minimal changes.

1.2 Technologies Used

Frontend

• HTML5: Used to structure and organize the web pages.
• CSS3: Used for designing and styling web pages to ensure a clean, responsive layout.
• JavaScript: Provides dynamic content handling and user interactivity — such as displaying signs, updating translated text, and handling form submissions without reloading the page.
• Bootstrap: Simplifies responsive design and layout alignment for various screen sizes and devices.
• AJAX (Asynchronous JavaScript and XML): Enables smooth communication between frontend and backend without full-page reloads.

Backend

• Python 3: The main programming language used to implement all backend logic, including sign–text mapping, translation handling, and text-to-speech features.
• Django Framework: A high-level Python web framework used to build and manage the entire web application. It follows the Model–View–Template (MVT) architecture for clean separation of logic, design, and data.
• pyttsx3 Library: Used for Text-to-Speech (TTS) conversion, allowing the system to produce speech output for translated text offline.
• OS Module: Handles file and path management, such as loading images and GIFs dynamically from directories.
• Django Template Engine: Renders dynamic HTML pages by integrating backend data directly into templates.
• smtplib & email.mime: Used to send feedback or support messages through the contact form to the administrator.

Database

• SQLite3: The default lightweight relational database used with Django to store predefined mappings between text, words, and corresponding sign-language media files.
• Django ORM: Provides an easy-to-use interface for database interactions without writing SQL queries directly.
• Migration System: Ensures smooth schema updates when models are modified.

Multimedia Resources

• Images and GIFs: Represent the visual elements of Indian Sign Language (ISL). These files are stored locally and are displayed dynamically during translation.
• Audio Files: Used to play pre-recorded or dynamically generated audio output.
• Static File Management: Django's static files system efficiently serves multimedia content and manages caching for faster performance.
• Media Directory Structure: Organized folder hierarchy for easy maintenance of gesture and audio resources.
• Cross-Browser Compatibility: Ensures smooth playback of multimedia elements across major browsers like Chrome, Firefox, and Edge.

2. FEASIBILITY ANALYSIS

2.1 Technical Feasibility

The Sign Language Translator project is technically feasible because it uses open-source, stable, and well-documented technologies. The backend is developed using Python and the Django framework, which provide a secure, modular, and scalable architecture for building web applications. Django's built-in features such as URL routing, ORM, and template handling make development efficient and maintenance easy.

The frontend is built using HTML5, CSS3, and JavaScript, ensuring compatibility with all modern browsers. The use of SQLite as a database ensures lightweight, serverless, and reliable data storage, suitable for both local and small-scale deployments.

2.2 Economic Feasibility

The project is economically feasible because all technologies and tools used in its development are free and open-source. Python, Django, SQLite, and text-to-speech libraries (like pyttsx3) do not require any licensing costs. Development and testing were performed using freely available IDEs such as PyCharm Community Edition or Visual Studio Code, further reducing expenses.

Additionally, the system can run efficiently on standard computer hardware without the need for high-end servers or GPUs, minimizing setup costs. Maintenance costs are also low, as updates and extensions can be handled through open-source libraries and frameworks. Therefore, the project is highly cost-effective and practical for both educational and institutional deployment.

2.3 Operational Feasibility

The Sign Language Translator system is designed with simplicity and user-friendliness in mind. It features an intuitive web-based interface that can be easily operated by users with basic computer knowledge. The input process and translation display are straightforward, requiring minimal training or technical expertise.

The backend processes are automated, ensuring that once the system is launched, it operates smoothly without manual intervention. Users can translate signs, view corresponding animations, and listen to speech outputs through simple button clicks. The system can be deployed locally or on an intranet, making it accessible in classrooms, training centers, or communication labs.

The system requires minimal maintenance — administrators can easily update sign-language content or modify translations through the Django admin panel. It supports cross-platform compatibility, running effectively on Windows, Linux, or macOS. The application can also be accessed via any modern web browser without requiring additional installations.

Additionally, the interface is responsive and accessible, ensuring smooth operation on both desktop and mobile devices. Built-in error handling provides informative prompts to guide users during incorrect input or missing data scenarios.

3. SYSTEM ANALYSIS

3.1 Existing System

In the current communication landscape, interaction between hearing-impaired individuals and those unfamiliar with sign language remains a major challenge. Traditionally, communication relies heavily on human interpreters, lip reading, or written notes, which are often inconvenient, slow, and limited in accuracy. Educational institutions and organizations may use printed sign language charts or instructional materials to teach communication basics, but these static resources lack interactivity and real-time engagement.

While some digital resources exist in the form of videos or images demonstrating sign language, they are generally designed for learning purposes rather than real-time translation. These systems often require manual searching for each sign, which makes the process inefficient for spontaneous communication. There is also no integrated platform that can dynamically translate text into corresponding sign-language gestures or vice versa. As a result, communication barriers continue to exist between the hearing and non-hearing communities, particularly in educational, professional, and public service environments. This highlights the need for a more interactive, automated, and user-friendly system that can facilitate instant translation between text and sign language.

Limitations of Existing System

- Manual Interpretation: Communication depends on human interpreters or written notes, which is slow and impractical in many real-life situations.
- Lack of Automation: Existing resources like printed guides and videos cannot perform automatic translation or interactive conversion between text and signs.
- Limited Accessibility: Most tools are not easily accessible to common users and require prior knowledge of sign language.
- Non-Interactive Learning: Available materials are static and do not provide hands-on practice or instant feedback.
- Absence of Unified Platform: There is no centralized application that combines text-to-sign, sign-to-text, and speech functionalities for Indian Sign Language (ISL).
- Dependency on Human Expertise: Users often rely on trainers or interpreters to facilitate communication, limiting independence.

3.2 Proposed System

The proposed system is a web-based Sign Language Translator designed to bridge the communication gap between hearing and speech-impaired individuals and those unfamiliar with sign language. It provides a two-way translation mechanism where users can convert text into corresponding sign-language gestures (displayed as images or GIFs) and optionally convert the translated text into speech output. The system focuses on accessibility, ease of use, and real-time interaction without the use of artificial intelligence or complex machine learning models.

The application is developed using Python as the core programming language and the Django framework as the web development platform. Django's Model–View–Template (MVT) architecture ensures clean separation between data handling, application logic, and presentation layers. The frontend of the system is built using HTML5, CSS3, and

JavaScript, creating an intuitive and user-friendly interface for communication. Users can input text or select predefined options, and the system dynamically fetches and displays the corresponding sign-language representations from a predefined database of images and GIFs.

In addition to text-to-sign translation, the system integrates a text-to-speech (TTS) module implemented using the pyttsx3 library. This enables the translated text to be converted into audible speech, facilitating better communication with individuals who rely on spoken language. The SQLite database stores mappings between text entries and their corresponding sign-language media files, ensuring fast retrieval and offline functionality.

The modular structure of the application allows for easy maintenance and scalability. The Django backend is responsible for managing user interactions, handling requests, retrieving data, and rendering appropriate responses. The frontend dynamically displays the translated results without reloading the entire page, providing a smooth user experience.

Furthermore, the system is designed to operate efficiently on standard hardware and can be deployed either locally or on an intranet. Since all technologies and resources used are open-source, the project remains cost-effective and easily adaptable for educational or institutional use. Future versions of the system can include features such as sign-to-text conversion, multi-language support, and mobile compatibility.

Advantages of Proposed System

• Bridges Communication Gap: Enables effective communication between hearing-impaired individuals and those unfamiliar with sign language by translating text into sign-language gestures.

• Two-Way Interaction: Provides both text-to-sign and optional text-to-speech conversion, making communication smoother for all users involved.

• User-Friendly Interface: Designed using standard web technologies such as HTML, CSS, and JavaScript to ensure simplicity, accessibility, and responsiveness for users with basic computer skills.

• Offline Accessibility: Since the system relies on predefined image and GIF libraries instead of online APIs or machine learning models, it can function without an active internet connection.

• Cost-Effective Solution: Utilizes open-source technologies (Python, Django, SQLite, pyttsx3), eliminating licensing or subscription costs and ensuring affordability for educational or institutional use.

• Modular and Scalable Architecture: Built using Django's MVT architecture, making the application easy to maintain, extend, and integrate with future features such as sign-to-text translation or multilingual support.

• Fast and Reliable Performance: The database-driven mapping of text to gestures ensures quick response times and smooth operation even on standard computing hardware.

• Inclusive and Educational Tool: Can serve both as a learning platform for individuals interested in sign language and as an assistive communication aid for people with hearing or speech disabilities.

4. SYSTEM REQUIREMENTS

4.1 Hardware Requirements

Processor: Intel Core i3 or equivalent
RAM: 4 GB (8 GB or more recommended)
Storage: Minimum 10 GB free disk space
Display: 1024 × 768 resolution monitor
Internet: Required for setup, updates, and optional email functionality
Input Device: Standard keyboard and mouse (webcam optional for gesture display testing)

4.2 Software Requirements

Operating System: Windows 10 or above / Linux (Ubuntu recommended)
Programming Language: Python 3.8 or higher
Backend Framework: Django (MVT architecture)
Frontend Technologies: HTML5, CSS3, JavaScript
Text-to-Speech Module: pyttsx3
Database: SQLite3 (default Django database engine)
Libraries/Modules Used: os, pyttsx3, sqlite3, django, pillow (for image handling)
Browser: Latest version of Google Chrome, Mozilla Firefox, or Microsoft Edge
IDE (Optional): VS Code, PyCharm, or any preferred text editor

5. SYSTEM DESIGN

5.1 System Architecture

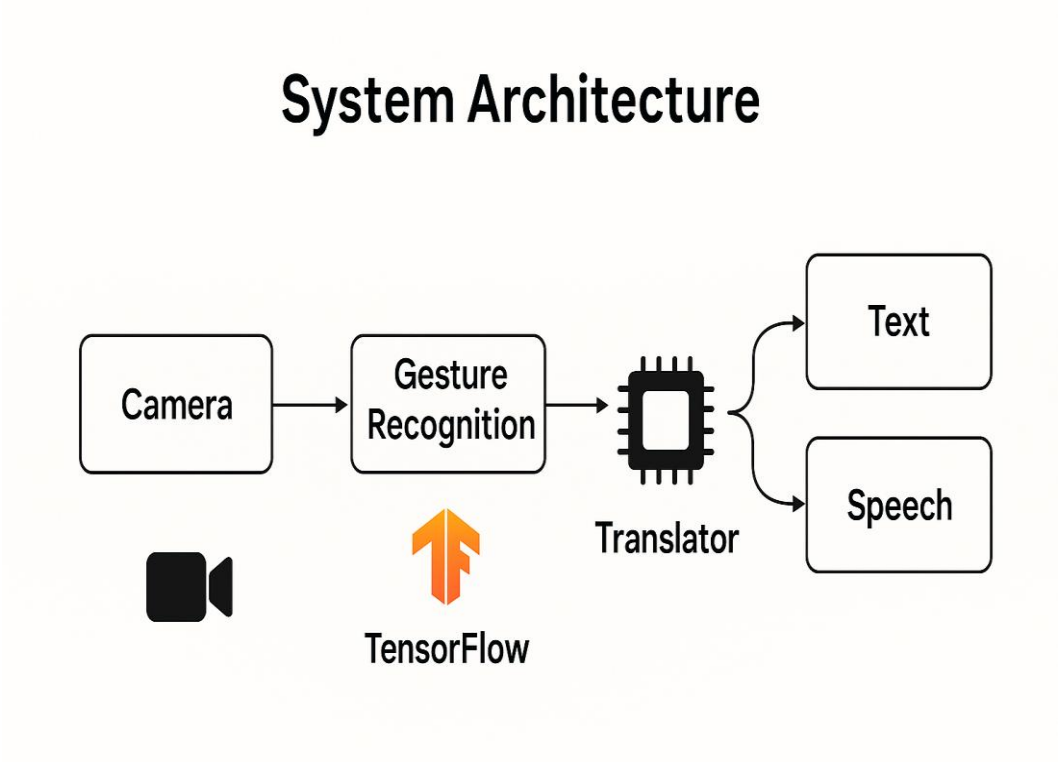

Figure 1 System Architecture

User Input via Web Interface

Users enter text or predefined phrases through a clean and interactive web interface built using HTML, CSS, and JavaScript.

The interface is simple, accessible, and designed to display the corresponding sign-language gestures (images or GIFs) in real time.

Django Backend as Central Controller

- The Django backend receives user input from the web interface and coordinates all internal processing.
- It acts as the central communication hub between the frontend, database, text-to-speech module, and media resources.
- The Model–View–Template (MVT) architecture ensures efficient separation of logic, data, and presentation, enhancing maintainability and scalability.

Sign Language Translation Module

- The backend uses a predefined mapping system to translate text into corresponding sign-language images or GIFs.
- Each character, word, or phrase is linked to a visual representation stored in the database or media directory.

- The translated gestures are displayed in sequence, simulating continuous sign communication for better understanding.

Text-to-Speech Integration

- The system integrates text-to-speech (TTS) functionality using the pyttsx3 library.
- After translation, the entered text can be converted into audible speech, allowing communication with hearing individuals.
- This feature is particularly useful in inclusive environments such as classrooms or workplaces.

Database Interaction

- The SQLite3 database stores text–gesture mappings, user information, and media metadata.
- The Django ORM (Object-Relational Mapper) efficiently handles database queries and data retrieval.
- This ensures smooth performance and fast access to required gesture resources.

Media Management System

- A structured media library stores all sign-language images and GIFs categorized alphabetically or by phrase.
- Django's static and media file management handles retrieval and rendering of these gesture files dynamically.
- The organized structure allows easy updates or addition of new gestures in the future.

Email Service for Communication (Optional)

- The system can include an email automation service using smtplib and email.mime libraries.
- It handles contact form submissions and automatically sends feedback or query notifications to the administrator.
- This ensures proper support and communication between users and system maintainers.

User Feedback Loop

- User messages or queries submitted via the contact form are routed through the email service.
- They can be stored or logged for admin review and future system improvement.
- This establishes a feedback mechanism that enhances user experience and support.

Output to User

- The frontend displays the translated sign-language gestures corresponding to the entered text.
- The TTS module simultaneously plays the audio version of the text, enabling clear communication.
- Users can visualize and hear the translated message in real time, ensuring smooth and inclusive interaction.

5.2 Sequence Diagram

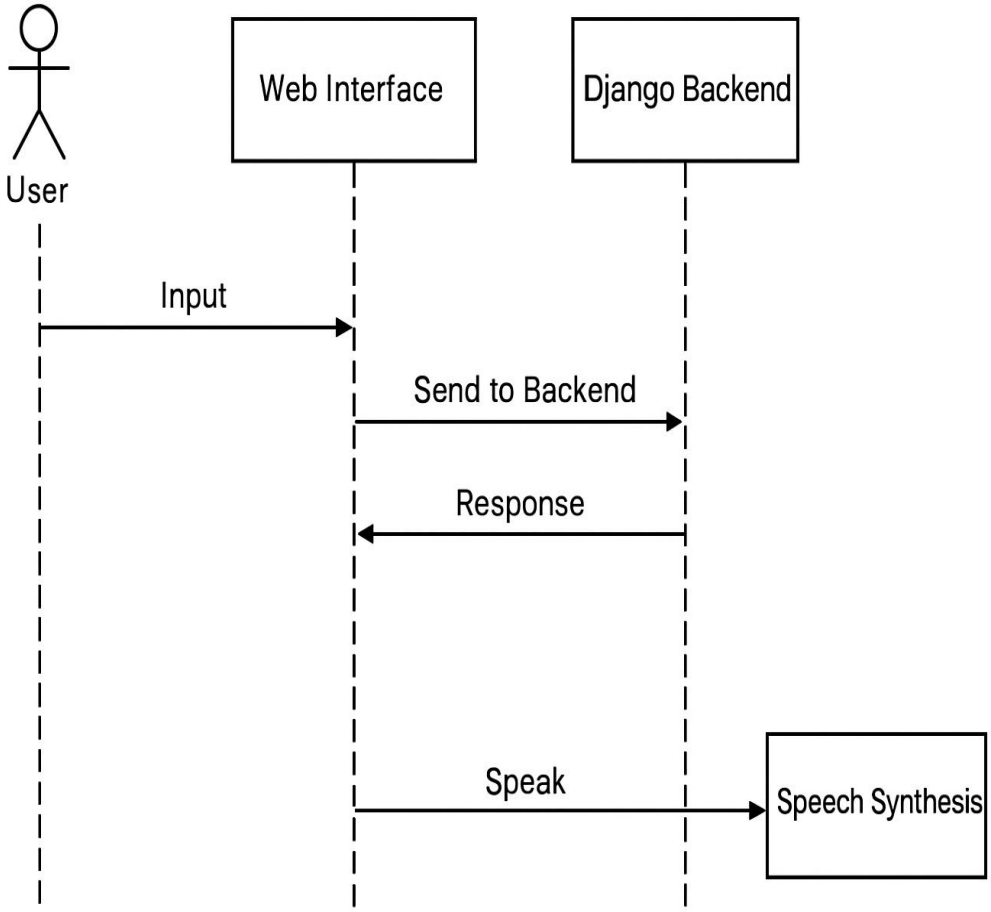

Figure 2 Sequence Diagram

User Interaction Starts the Process
The user begins by entering text or selecting predefined phrases through the web interface. This input serves as the starting point for translation into sign language.

Frontend Sends Data to Backend
The HTML/CSS/JavaScript frontend captures the user's text input and sends it to the Django backend via HTTP requests.
The request is handled asynchronously to ensure a smooth and responsive user experience.

Backend Processes Request
The Django backend acts as the main controller. It receives the user's input, validates it, and determines the corresponding sign-language gesture files stored in the system.
The backend ensures that each character, word, or phrase is correctly mapped to its associated sign representation.

Backend Retrieves Gesture Data
The backend retrieves the appropriate gesture images or GIFs from the SQLite3 database or static media directory based on the input text.
If the text contains multiple words or characters, the system arranges the gestures in sequence for continuous visual translation.

Text-to-Speech (TTS) Conversion
Simultaneously, the pyttsx3 module converts the entered text into audible speech.
This allows hearing individuals to understand what the non-hearing user intends to communicate.

Backend Sends Combined Response
Once processing is complete, the backend compiles the gesture sequence and audio output link into a structured response.
This data is then sent back to the frontend through Django's templating system or AJAX-based response.

Frontend Displays Output to User
The web interface displays the sign-language gestures in animated or sequential format and plays the corresponding speech output.
Users can visually follow the gesture translation and listen to the spoken text in real time, ensuring effective two-way communication.

5.3 Class Diagram

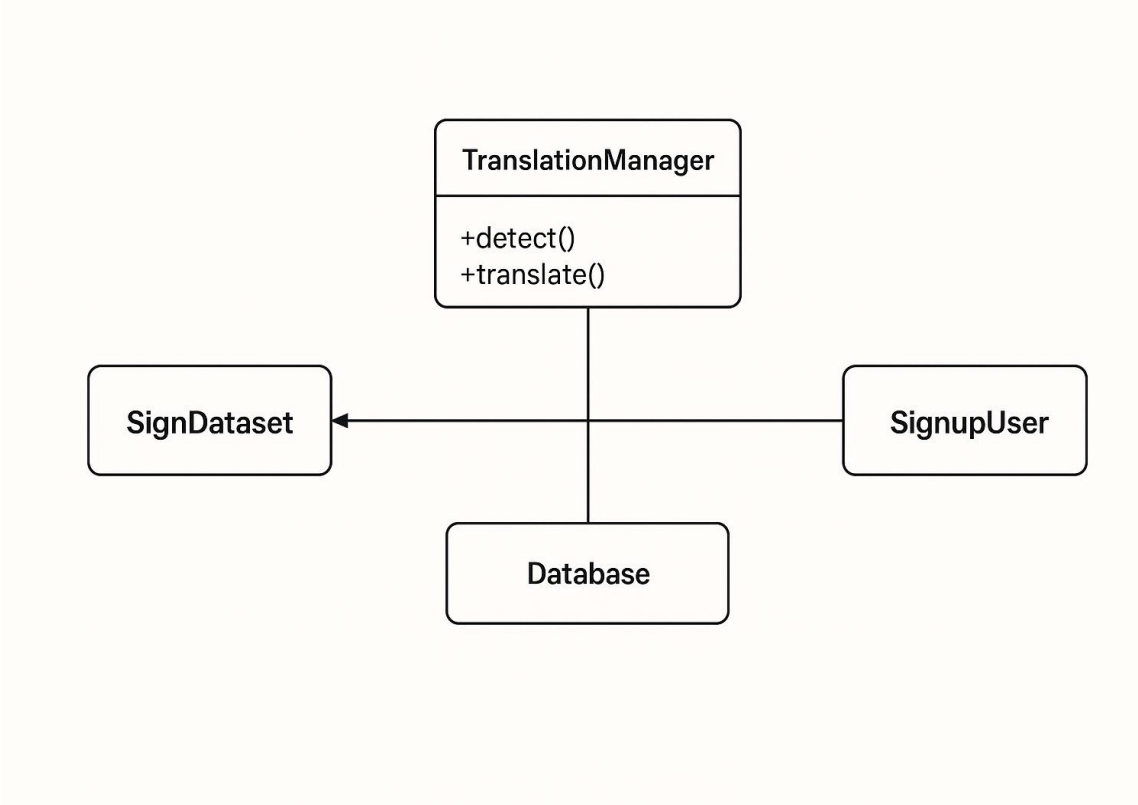

Figure 3 Class Diagram

UserInput Class

- Represents: The input provided by the user through the web interface.
- Attributes:
  - text_input *(string)* – The text entered by the user for translation.
  - selected_phrase *(string)* – Optional predefined phrase chosen from a list.
- Methods:
  - get_input() – Captures and returns user input for processing.

Gesture Class

- Represents: The stored sign-language elements used for translation.
- Attributes:
  - gesture_id *(integer)* – Unique identifier for each gesture.
  - gesture_name *(string)* – The name or description of the gesture.
  - file_path *(string)* – Path to the image or GIF representing the gesture.
- Methods:
  - get_gesture() – Retrieves the gesture file corresponding to the input text.
  - update_gesture() – Allows modification or addition of new gestures (admin only).

Translator Class

- Acts as: The central controller for translation logic.
- Attributes:
  - input_text *(string)* – Stores text received from the user.
  - translated_gestures *(list)* – Contains references to gesture files matched with input.
  - audio_output *(string)* – Stores path or stream of the generated speech output.
- Methods:
  - translate_text() – Converts text into a sequence of corresponding sign-language gestures.
  - generate_audio() – Produces audible speech using the pyttsx3 text-to-speech module.
  - display_translation() – Sends translated results (gestures and audio) to the frontend.

DatabaseManager Class

- Handles: Database operations for retrieving and updating records.
- Attributes:
  - connection – SQLite database connection instance.
  - cursor – SQL cursor for executing queries.
- Methods:
  - fetch_mapping() – Retrieves sign-language mappings for given text inputs.
  - insert_record() – Adds new gesture or user data into the database.
  - update_record() – Updates existing data entries as needed.

Admin Class

- Represents: The administrator responsible for managing the system's data and users.
- Attributes:
  - admin_id *(integer)* – Unique identifier for the admin.
  - name *(string)* – Administrator's name.
  - email *(string)* – Contact email address.
- Methods:
  - manage_users() – Handles user registration and deletion.
  - manage_gestures() – Adds, modifies, or removes gestures from the database.
  - view_feedback() – Displays user feedback and contact messages.

Relationships

- Translator has-a → UserInput (receives input from the user).
- Translator has-a → Gesture (uses gesture data for translation).
- Translator has-a → DatabaseManager (retrieves and stores mappings).
- Admin manages → Gesture (adds or edits gesture records).
- Admin manages → UserInput/User (monitors system activity).

5.4 Use Case Diagram

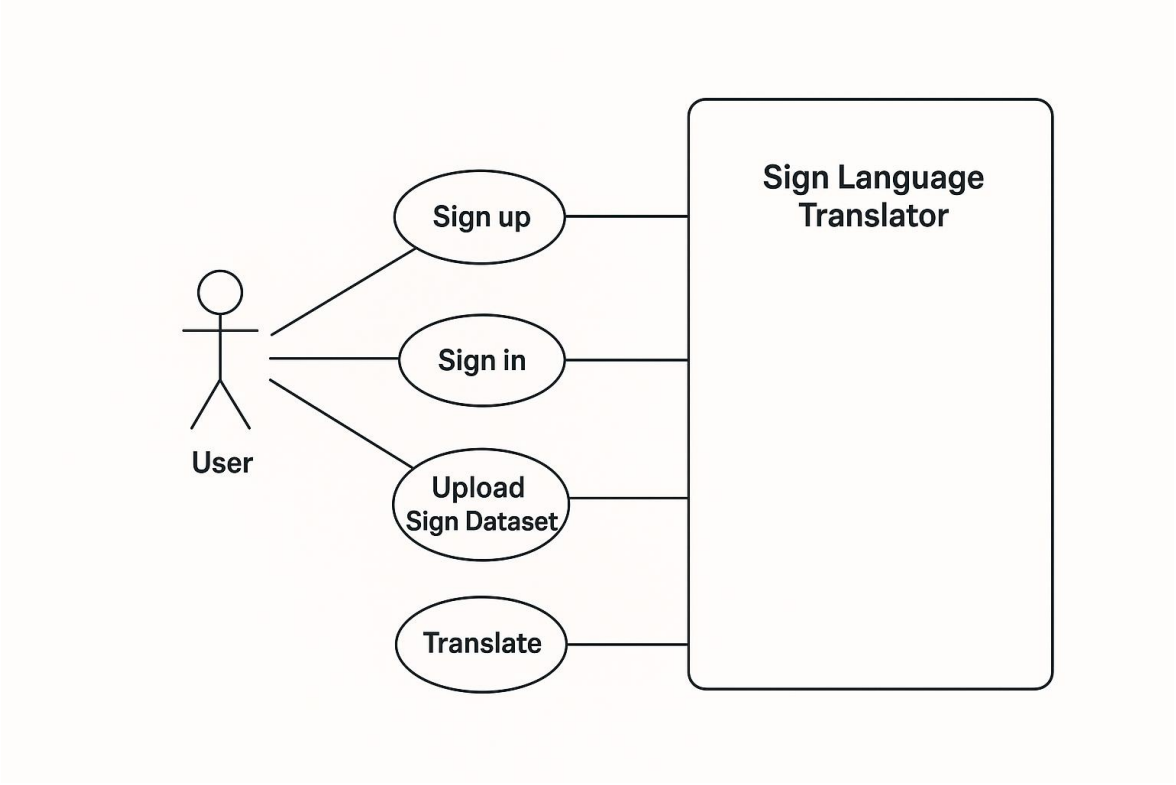

Figure 4 Use Case Diagram

Primary Actor – User

- The diagram identifies a single primary external actor: the User (a hearing or speech-impaired person, or anyone using the translator).
- The user interacts with the system through various use cases related to translation, learning, and communication.

System Boundary – Sign Language Translator System

- Represents the entire *Sign Language Translator* platform enclosed within a defined system boundary.
- All user-accessible functionalities such as text translation, speech conversion, and feedback submission are represented as use cases within this boundary.

Use Case: Enter Text or Select Phrase

- The user inputs text manually or selects a predefined phrase through the web interface.
- This use case initiates the sign-language translation process.

Use Case: View Sign-Language Translation

- Once text input is submitted, the system retrieves the corresponding sign-language images or GIFs and displays them in sequence.
- This allows the user to visualize how the input text would be represented using sign language.

Use Case: Listen to Speech Output

- The system uses the Text-to-Speech (TTS) module to convert the input text into audible speech.
- This use case helps bridge communication between hearing and non-hearing individuals by providing audio playback of the translated message.

Use Case: Learn Sign Language

- The user can browse available sign-language alphabets, words, and gestures stored in the system.
- This feature serves as a learning module, allowing users to practice and understand Indian Sign Language (ISL) symbols effectively.

Use Case: Submit Feedback or Query

- The user can submit feedback, questions, or support requests through the contact form on the website.
- The system handles these submissions using automated email services via smtplib and email.mime.

Use Case: Admin Management (Secondary Actor – Admin)

- A secondary actor, Admin, manages system data such as gesture records, user messages, and media files.
- Admin can add, update, or delete gestures, and monitor user feedback for system enhancement.

5.5 Activity Diagram

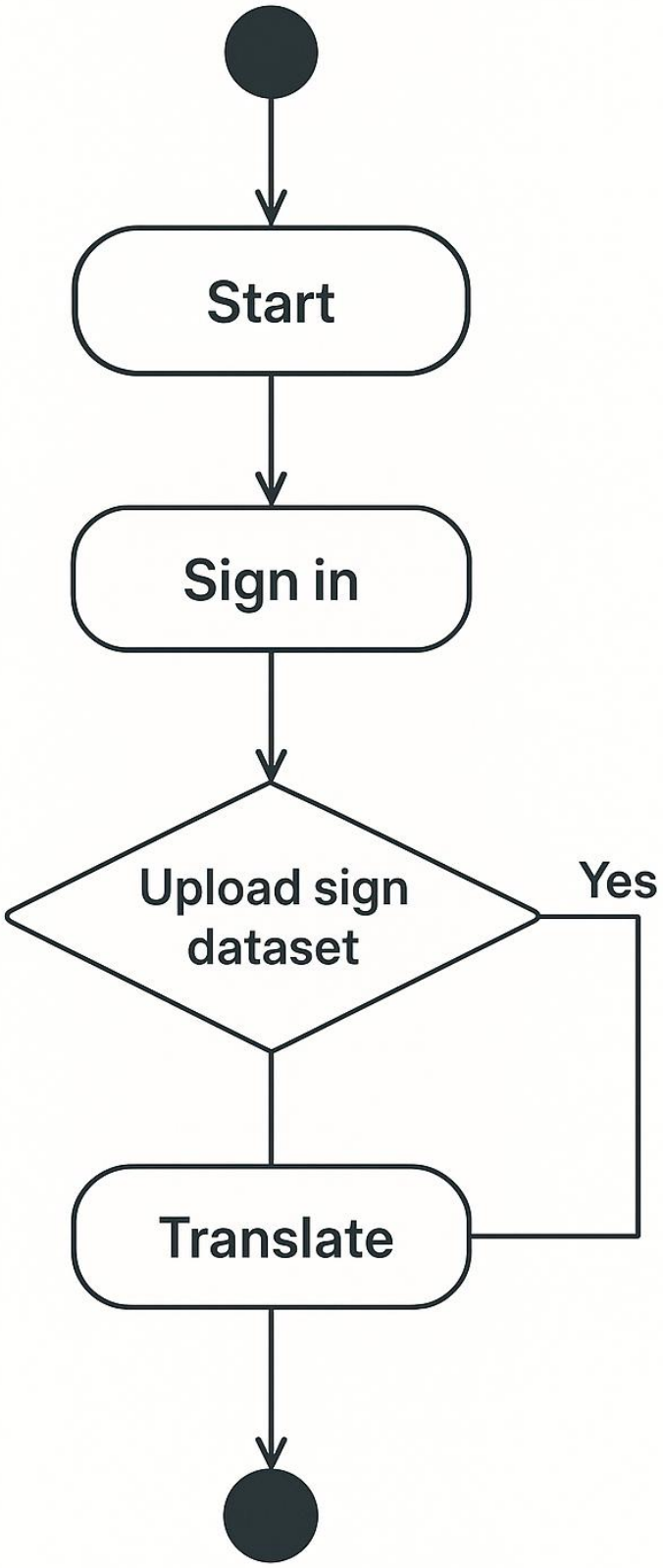

Figure 5 Activity Diagram

Start Node
The process begins when the user accesses the Sign Language Translator system through the web interface.

Enter Text or Select Phrase
The user provides input by typing text or selecting a predefined phrase from the interface.
This input serves as the starting point for the translation process.

Submit Input
Once the user finishes entering the text, the input is submitted to the Django backend for processing.
The backend validates the text and prepares to retrieve corresponding sign-language representations.

Display Translation (Decision Node)

- A decision is made based on whether the sign-language translation is successfully retrieved.
- Yes: Continue to display the translation results and generate speech output.
- No: Skip translation display and show a message indicating that the sign or phrase is not available**.**

View Sign-Language Output
If translation is available, the system retrieves the mapped images or GIFs and displays them sequentially to represent the user's input in sign language.

Listen to Speech Output
Simultaneously, the Text-to-Speech (TTS) module converts the input text into audio output, allowing hearing individuals to understand the communication.

Learn Sign Language (Optional)
The user can optionally explore the learning module, which contains alphabets, words, and common phrases in Indian Sign Language (ISL).
This helps users practice and improve their sign-language knowledge.

Submit Feedback or Query
The user may send feedback, queries, or support requests through the system's contact form.
These are handled via an email service (smtplib and email.mime) that forwards messages to the administrator.

End Node
Marks the completion of the activity flow, ending the user's session or returning to the homepage for a new translation.

5.6 Data Flow Diagram

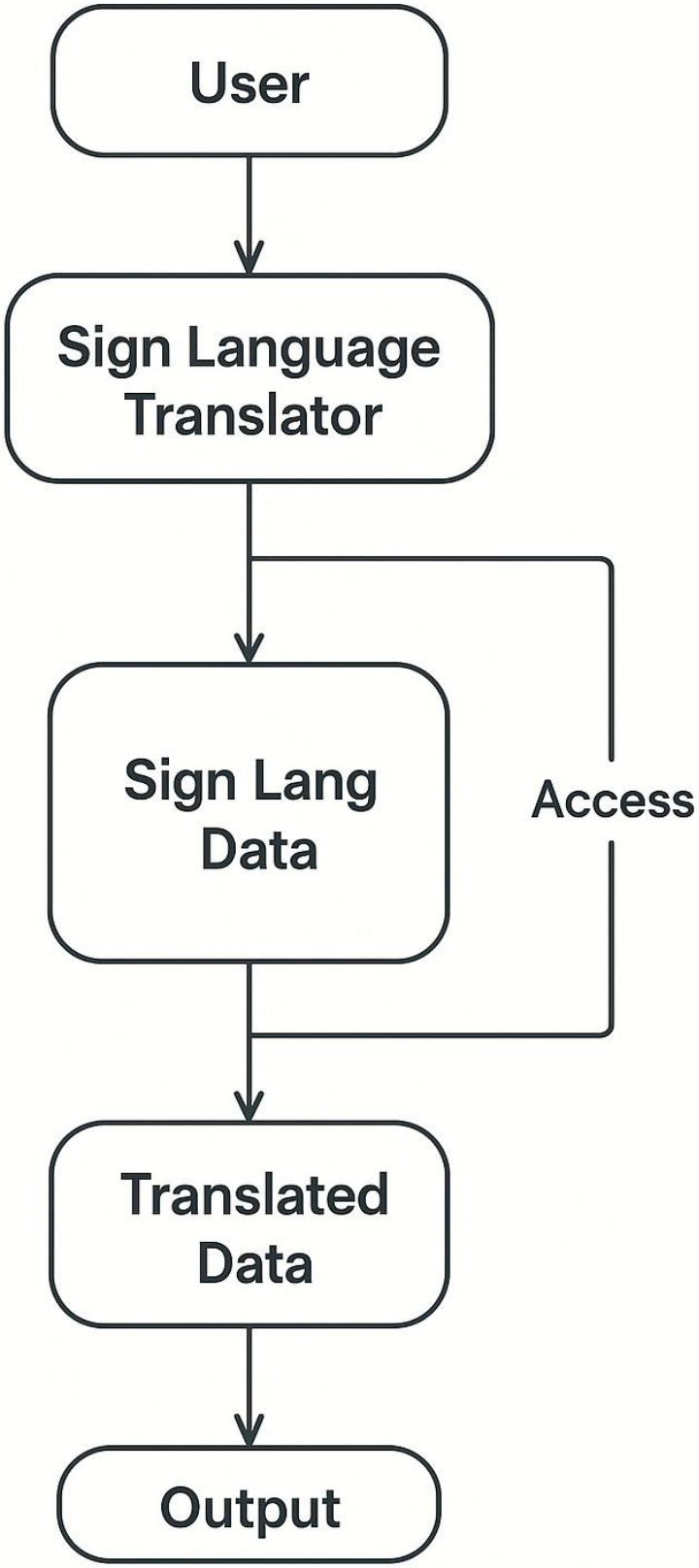

Figure 6 Data Flow Diagram

External Entity – User

- The User is the primary external entity who interacts with the *Sign Language Translator* system.
- The user provides input text or selects predefined phrases for translation into sign language.

Process 1.0 – Submit Input

- This process captures the user's input (text or phrase) through the web interface.
- The input is then forwarded to the Django backend for processing.
- Data includes the entered text and optional phrase selection, which are required for translation and speech output.

Process 2.0 – Generate Translation

- This process retrieves the corresponding sign-language images or GIFs based on the user's input.
- It uses predefined mappings stored in the SQLite database to convert text into sign-language representation.

Data Store 3.0 – Input Data

- Stores all submitted text inputs and user details for reference or future enhancement of the system.
- Acts as a central repository of raw user input and logs interactions for potential analysis or debugging.

Process 3.0 – Display Results

- Combines the translated sign-language gestures and speech output for display on the frontend.
- Returns the output to the user interface, ensuring that both visual (sign) and auditory (speech) results are synchronized.

Data Store 4.0 – Gesture Repository

- Contains preloaded Indian Sign Language (ISL) gestures, alphabets, words, and common phrases.
- Each gesture is stored as an image or GIF file linked to corresponding text entries in the database.
- Supports the translation process by providing accurate visual content for display.

Process 4.0 – Handle Feedback

- This process manages user feedback, queries, or support requests submitted through the contact form.
- It triggers the email automation service (smtplib, email.mime) to forward messages to the administrator for review.

Data Flow Paths

- Arrows in the diagram represent the logical flow of data between the User, Processes, and Data Stores.
- For example:
  - The Submit Input process sends data from the User to Generate Translation.
  - Generate Translation retrieves gesture data from the Gesture Repository and sends combined results to Display Results.
  - Display Results outputs the translation and audio back to the User interface.
  - Handle Feedback sends communication data to the Admin via the email service.

5.7 E-R Diagram

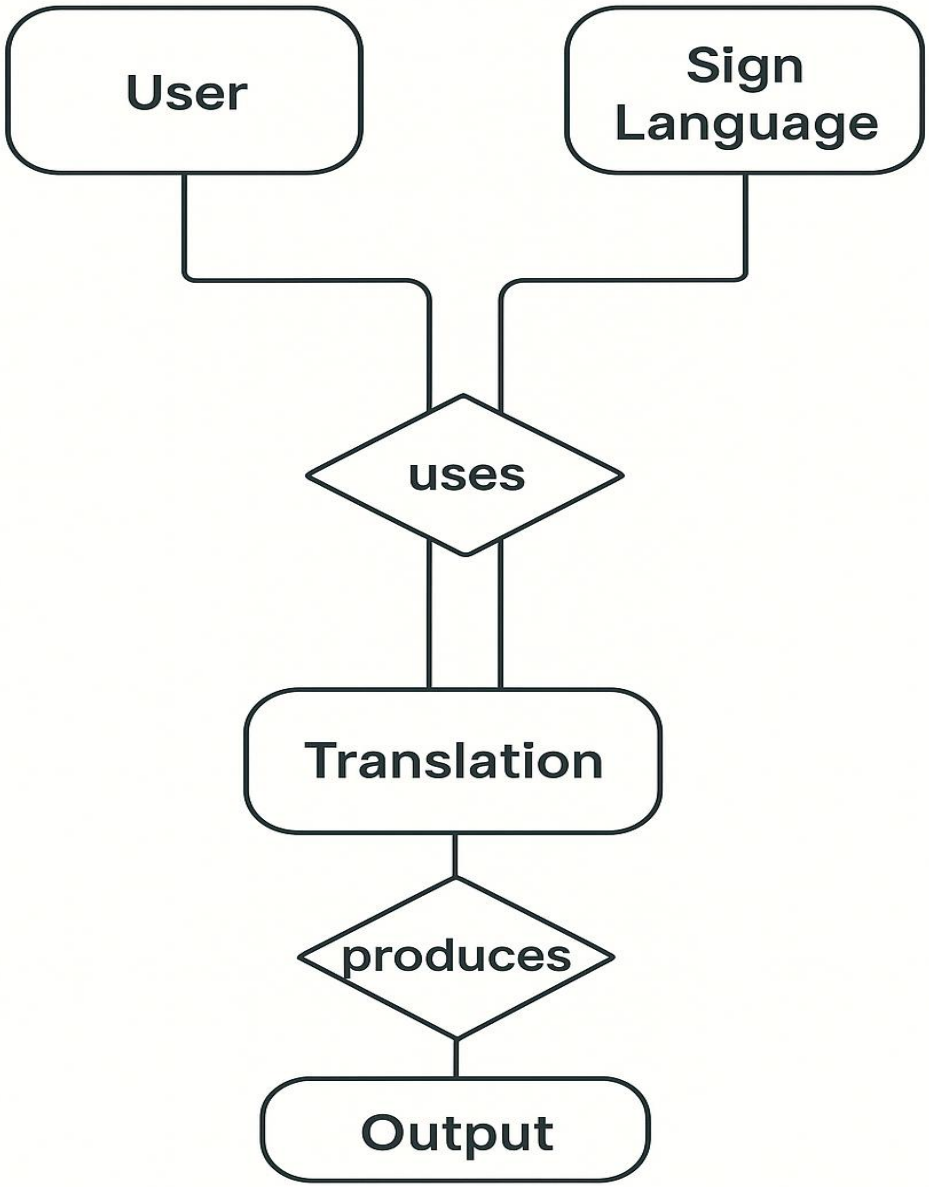

Figure 7 E.R. Diagram

Entity: UserInput

-
- Description:
  Represents the text or phrase entered by the user through the web interface for translation.
- Attributes:
  - input_id – Unique identifier for each user input.
  - text_input – The text entered by the user.
  - selected_phrase – Optional predefined phrase chosen from the interface.
- Relationships:
  Connected to Translation and AudioOutput entities, as both depend on the user's text input.

Entity: Translation

- Description:
  Represents the sign-language translation generated from the user's text input.
- Attributes:
  - translation_id – Unique identifier for each translation record.
  - gesture_sequence – The list or file paths of gestures (images/GIFs) used to represent the text.
- Relationships:
  Linked to UserInput (each input produces one translation).
  Also connected to the GestureLibrary, which provides gesture data for mapping.

Entity: GestureLibrary

- Description:
  Stores the predefined set of Indian Sign Language (ISL) gestures in the form of images or GIFs.
- Attributes:
  - gesture_id – Unique identifier for each gesture.
  - gesture_name – The name or description of the gesture.
  - file_path – The storage path of the gesture's image or animation file.
- Relationships:
  Linked to Translation, as gestures are fetched from this entity to generate visual output.

Entity: AudioOutput

- Description:
  Represents the speech output generated using the text-to-speech (TTS) module.
- Attributes:
  - audio_id – Unique identifier for each audio record.
  - audio_file – The generated or stored speech file corresponding to the text input.
- Relationships:
  Connected to UserInput, since the text entered by the user forms the basis of speech output.

Entity: Feedback

- Description:
  Represents user feedback, queries, or contact requests submitted through the system's contact form.
- Attributes:
  - feedback_id – Unique identifier for each feedback entry.
  - user_email – Email address of the user submitting feedback.
  - message – Content of the query or feedback.
- Relationships:
  Linked to UserInput (feedback may relate to a particular translation session).
  Also connected to the Admin entity for review and response.

Entity: Admin

- Description:
  Represents the system administrator who manages users, gestures, and system content.
- Attributes:
  - admin_id – Unique identifier for the administrator.
  - name – Administrator's name.
  - email – Admin's contact email.
- Relationships:
  - Manages → GestureLibrary (can add or update gesture files).
  - Reviews → Feedback (receives and responds to user messages).

Relationships

• UserInput → Translation: One-to-one relationship (each input produces one translation).
• UserInput → AudioOutput: One-to-one relationship (each input may generate one speech output).
• Translation → GestureLibrary: One-to-many relationship (a translation may use several gesture files).
• UserInput → Feedback: One-to-many relationship (a user can submit multiple feedback messages).
• Admin → GestureLibrary / Feedback: One-to-many relationship (admin manages gestures and reviews feedback).
• GestureLibrary → MediaFiles: One-to-many relationship (each gesture may have multiple associated image or GIF files for animation).
• Admin → UserAccounts: One-to-many relationship (admin can create, update, or manage multiple user accounts).
• Translation → AudioOutput: One-to-one relationship (each translation generates a corresponding audio output for accessibility).

6. IMPLEMENTATION

6.1 Module Implementation

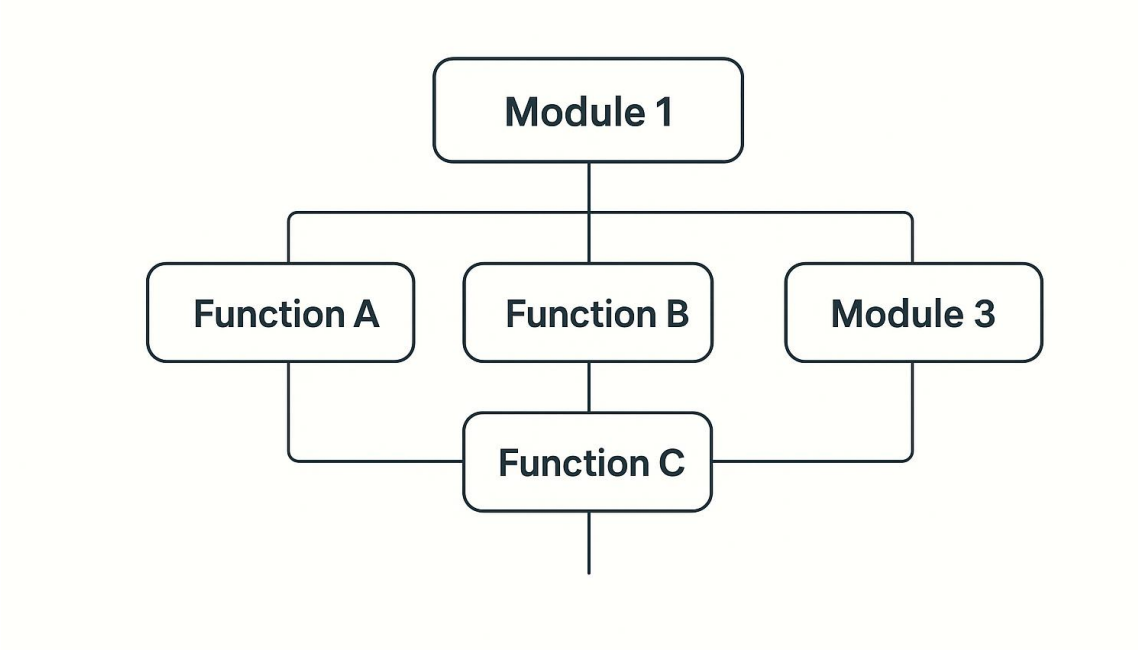

Figure 8 Module Implementation

Input Module (UserInput)

Purpose:

Collects user text or predefined phrase inputs through a simple, accessible web interface for translation.

Functions:

- Accepts user-entered text or selected predefined phrases.
- Validates the input to ensure it matches supported words or phrases.
- Sends the validated data to the backend for gesture translation and speech synthesis.
- Provides instant error messages or suggestions for unsupported words.
- Maintains a smooth user experience through AJAX-based dynamic requests.
- Ensures accessibility compliance for visually or hearing-impaired users.

Technology Used:

HTML5, CSS3, JavaScript (frontend), Django (backend).

Translation Module (Translator)

Purpose:

Converts the user's text input into its equivalent sign-language representation.

Functions:

- Maps each character, word, or phrase to corresponding gesture images or GIFs.
- Displays the gesture sequence dynamically on the frontend in real-time.
- Integrates with the Text-to-Speech Module to generate parallel speech output.

- Ensures accurate timing between sign display and spoken output.
- Caches frequently used gestures for faster translation.
- Allows scalability for adding more languages or gesture datasets in the future.

Technology Used:
Python, Django, SQLite3, HTML5, CSS3, JavaScript.

Gesture Management Module (GestureLibrary)
Purpose:
Stores and manages Indian Sign Language (ISL) gesture data.
Functions:

- Maintains a structured repository of gesture images and GIFs.
- Supports admin operations for adding, updating, or deleting gesture records.
- Ensures quick retrieval of gesture files during translation.
- Uses optimized indexing for faster media access.
- Provides version control for updated gesture representations.
- Links each gesture with its corresponding text or phrase entry in the database.

Technology Used:
Python, Django ORM, SQLite3.

Text-to-Speech Module (AudioOutput)
Purpose:
Converts textual input into speech output to aid communication between hearing and non-hearing users.
Functions:

- Uses the pyttsx3 library to synthesize speech from input text.
- Generates and plays an audio file for each translation session.
- Provides options for adjusting voice rate, tone, and volume.
- Supports multiple voice outputs depending on user preference.
- Ensures offline speech synthesis without requiring an internet connection.
- Integrates seamlessly with frontend audio playback components.

Technology Used:
Python, pyttsx3, Django.

Learning and Content Module (SignLearningContent)
Purpose:
Displays educational materials for learning Indian Sign Language (ISL) alphabets, words, and gestures.
Functions:

- Fetches static and dynamic learning content from the database.
- Presents categorized topics such as alphabets, greetings, and common phrases.
- Acts as a supplementary learning tool for both users and beginners.
- Enables multimedia-based tutorials using videos, GIFs, and illustrations.
- Tracks user learning progress and engagement (optional future feature).
- Offers simple navigation between learning topics for better user experience.

Technology Used:
Django Templates, HTML5, CSS3, JavaScript, SQLite3.

Database Module

Purpose:

Manages storage and retrieval of user data, gestures, translations, and feedback records.

Functions:

- Maintains normalized relational tables for all entities defined in the ER diagram.
- Supports insertion, updating, deletion, and querying operations.
- Ensures data consistency through Django ORM transactions.
- Provides secure data handling using Django's built-in model validations.
- Supports backup and recovery operations for gesture and content data.
- Enables easy migration for scaling to PostgreSQL or MySQL in the future.

Technology Used:

SQLite3 (default Django database).

Email and Contact Module

Purpose:

Handles user feedback and query submissions through the contact form**.**

Functions:

- Accepts and validates messages sent from users.
- Sends automated notifications to the system administrator using the email service.
- Records feedback for later review and system improvement.
- Filters spam or duplicate messages using basic validation logic.
- Provides confirmation messages to users after successful submission.
- Maintains a log of all contact form interactions for audit purposes.

Technology Used:

Python, Django, smtplib, email.mime.

Admin Module

Purpose:

Provides administrative control over gesture management, content updates, and user feedback.

Functions:

- Allows the admin to manage gesture data and learning content.
- Monitors user feedback and system activity logs.
- Uses Django's built-in admin interface for secure and efficient management.
- Grants role-based permissions for content editors and moderators.
- Provides a dashboard view for system analytics and usage statistics.
- Enables real-time content updates without requiring code modification.

Technology Used:

Django Admin Interface, Python, SQLite3.

6.2 SDLC

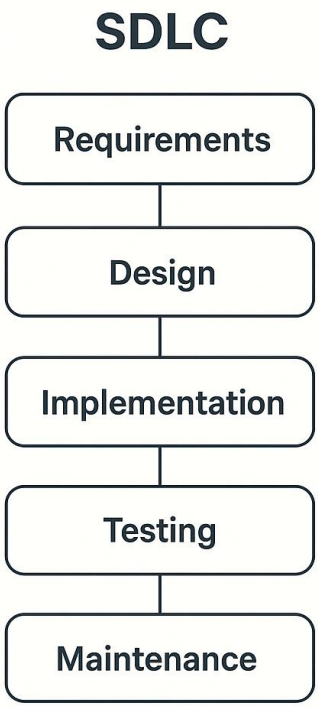

Figure 9 SDLC

Requirements

- The process began with gathering both functional and non-functional requirements for the Sign Language Translator system.
- The requirements focused on:
  - Enabling users to translate text or phrases into Indian Sign Language (ISL) gestures.
  - Providing an option for audio output using text-to-speech.
  - Supporting admin management for gesture uploads and content updates.
  - Ensuring an easy-to-use interface for both hearing and hearing-impaired users.

Design

- In this phase, the overall architecture of the system was defined.
- Major design elements included:
  - The web interface built with HTML, CSS, and JavaScript for input and display.
  - The Django MVC framework to manage backend logic, routing, and data handling.
  - The database schema (ER diagram) for organizing user input, gesture library, and feedback.
  - Selection of modules like the Translator, Text-to-Speech, and Gesture Management modules.

Implementation

- The actual coding and development were carried out in this phase.
- Modules implemented included:
  - Input Module for capturing text and phrases.
  - Translation Module for gesture mapping.
  - Text-to-Speech Module for converting text into audio.
  - Learning Module for educational sign-language content.
  - Admin and Email Modules for management and communication.
- The application was built using Python, Django, HTML/CSS/JavaScript, and SQLite.
- All components were integrated following the Model–View–Template (MVT) architecture for clear separation of logic and design.

Testing

- Each module and the integrated system were rigorously tested to ensure full functionality.
- Types of testing performed included:
  - Unit Testing: Verified each module's independent behavior.
  - Integration Testing: Ensured all modules worked seamlessly together.
  - User Interface Testing: Checked frontend usability and accessibility.
  - Functional Testing: Validated translation accuracy and audio playback.
- The testing phase ensured the system was stable, accurate, and user-friendly.
- Performance and stress tests were also conducted to ensure consistent operation under multiple user requests.

Deployment

- Once testing was complete, the application was deployed on a local or cloud-based server.
- The Django development server was used for local testing and could be extended to platforms like PythonAnywhere or Heroku for public access.
- Configuration ensured all modules (translation, audio, admin) operated efficiently in the hosted environment.
- Environment variables and security configurations (like SECRET_KEY and SMTP settings) were properly managed to ensure safe deployment.

Maintenance

- After deployment, maintenance focused on resolving any bugs, optimizing performance, and improving content.
- The system was designed for easy updates, allowing new gestures or phrases to be added through the admin panel.
- Future improvements may include:
  - Expanding to multiple sign languages.
  - Adding mobile compatibility for Android/iOS.
  - Enhancing performance with better database and caching strategies.
- Regular monitoring and user feedback collection ensure the system remains up-to-date and user-centric over time.

6.3 Waterfall Model

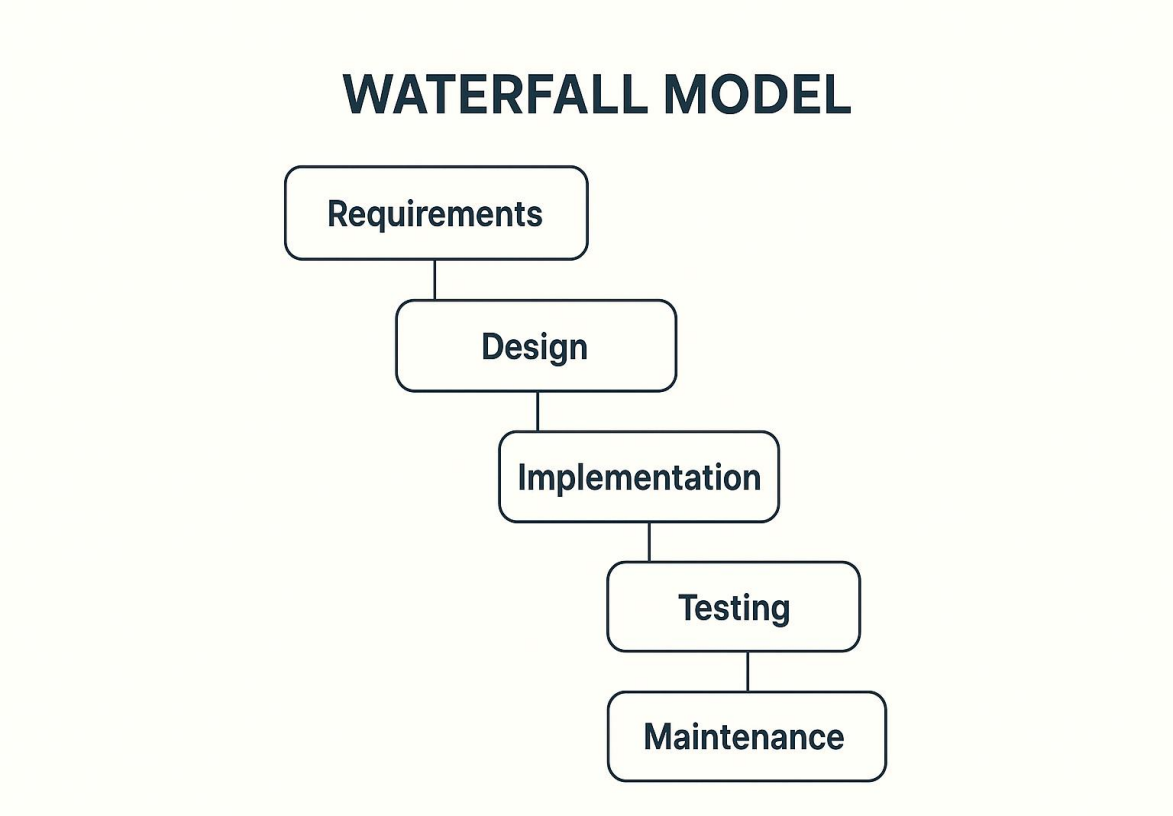

Figure 10 Waterfall Model

Requirement Analysis

- Understand user needs:
  Identify requirements from hearing-impaired users, educators, and general users who wish to learn or communicate using sign language.
- Define key functions:
  - Text or phrase input by users.
  - Conversion of text into Indian Sign Language (ISL) gestures.
  - Generation of audio output using text-to-speech (TTS).
  - Admin management for gestures and content updates.
  - Feedback or contact form for user communication.
- Identify required data:
  - Predefined ISL gesture dataset (images/GIFs).
  - Phrase-to-gesture mappings for translation.
- Plan system integration:
  Ensure smooth communication between frontend (web interface) and backend (Django server).
- Determine non-functional requirements:
  Define performance, scalability, accessibility, and response time goals to ensure the system remains efficient under real-world usage.

System Design

- Frontend Design:
  - Create responsive user interface using HTML5, CSS3, and JavaScript.
  - Develop separate pages for translation, learning, and contact forms.
- Backend Architecture:
  - Implement modular Django-based MVC structure.
  - Include dedicated apps for translation, user management, and contact handling.
- Database Design:
  - Define schema for gestures, users, translations, and feedback using SQLite3.
- Module Planning:
  - Translation (text → sign gestures)
  - Audio generation (text → speech)
  - Gesture management (admin uploads and updates)
  - Feedback handling (user messages sent to admin)
- Security Design:
  Implement basic authentication, form validation, and CSRF protection to ensure safe and secure user interactions.

Implementation

- Frontend Implementation:
  - Build interactive forms and dynamic content rendering using JavaScript.
  - Integrate gesture animations for smoother visual transitions.
- Backend Implementation:
  - Develop Django views and APIs for translation, feedback, and admin functions.
  - Implement TTS functionality using pyttsx3.
  - Store and retrieve gestures from the SQLite database dynamically.
- Admin Setup:
  - Use Django Admin for gesture, content, and feedback management.
- Code Optimization:
  Apply reusable templates, modular Python scripts, and caching techniques to improve code readability and execution speed.

Integration and Testing

- Integration:
  - Connect the frontend to backend APIs using AJAX or Fetch API calls.
  - Ensure gesture files load correctly and TTS output plays without delay.
  - Verify synchronization between sign display and audio output for a seamless user experience.
- Testing:
  - Perform unit testing for individual modules.
  - Conduct integration testing for complete translation flow (text → gesture → speech).

  - Validate system reliability across different browsers and devices.
  - Debug any issues related to animation display, form submission, or data retrieval.
  - Perform user acceptance testing (UAT) with sample users to ensure usability and accessibility standards are met.
- Regression Testing:
  After any code updates, retest the system to confirm that existing functionalities remain unaffected.

Deployment

- Local Deployment:
  - Run the Django server on localhost for testing and demonstration.
- Production Deployment:
  - Use a WSGI server such as Waitress or Gunicorn for deployment.
  - Optionally host on cloud platforms like PythonAnywhere, Heroku, or AWS.
- Post-deployment Checks:
  - Test the translation output, speech generation, and feedback email delivery.
  - Ensure responsiveness and accessibility across devices.
- Security Configuration:
  Set environment variables for sensitive data (SECRET_KEY, email credentials) and apply HTTPS for secure deployment.

Maintenance

- Content Updates:
  - Regularly update gesture datasets, add new words, and improve visual quality.
- Bug Fixes:
  - Monitor user feedback and correct any functional or interface issues.
- System Enhancements:
  - Extend support for multiple sign languages (e.g., ASL, BSL).
  - Improve translation speed and optimize database queries.
  - Add offline mode or mobile app integration for broader accessibility.
- Performance Monitoring:
  Track system usage metrics and response times periodically to ensure continued efficiency and scalability**.**

6.4 Sequential Model

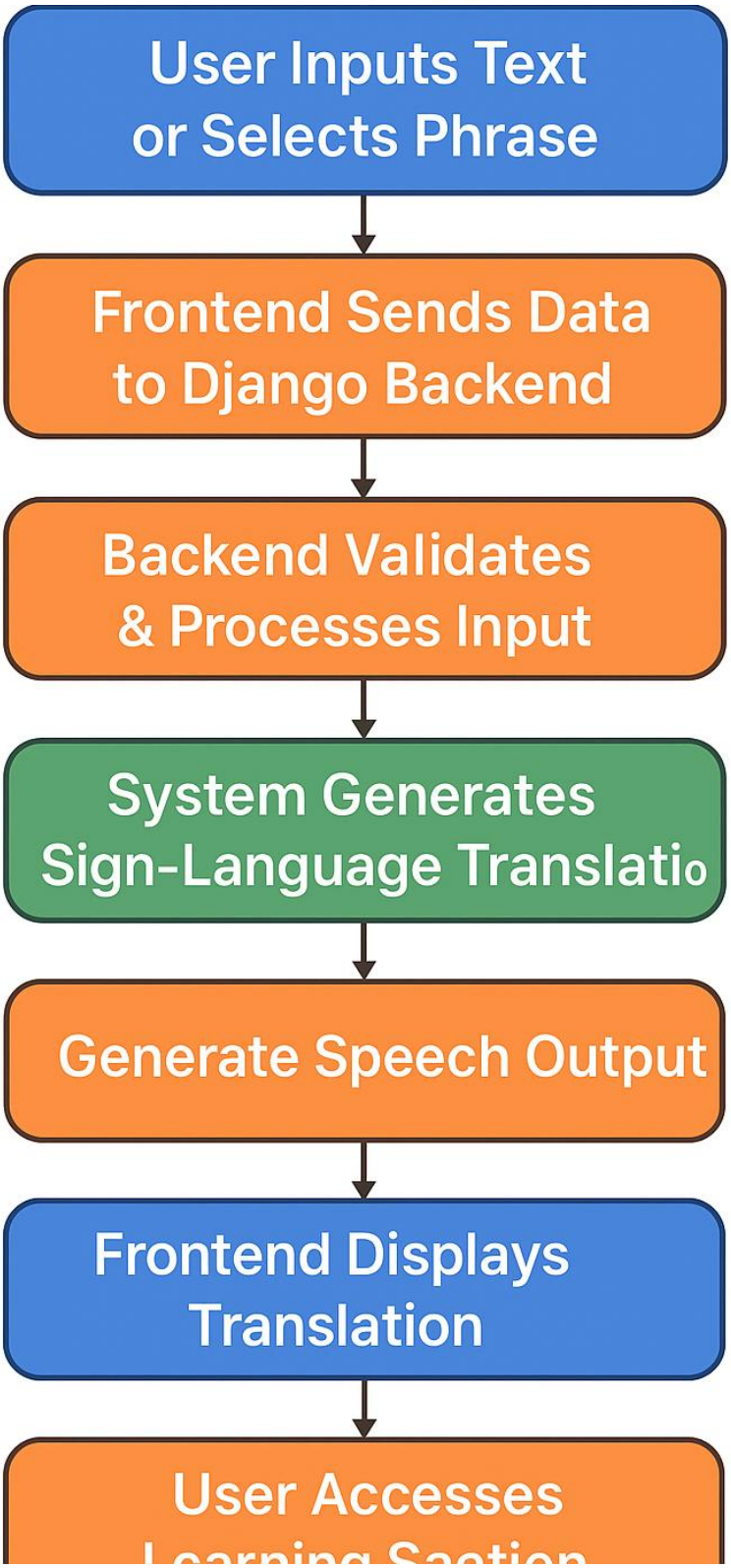

Figure 11 Sequential Model

User Accesses Web Portal

- This is the entry point to the system.
- The web interface is designed for both hearing-impaired users and those learning sign language.
- The home page provides options for text translation, learning ISL, and contacting the admin.
- The system performs a quick initial load of gesture and content data to ensure faster access to core functionalities.

User Inputs Text or Selects Phrase

- The user enters custom text or selects a predefined phrase from the dropdown.
- This input acts as the source for translation and speech output.
- The interface also provides instant suggestions or autocomplete for common words and phrases to improve usability.

Frontend Sends Data to Django Backend

- The frontend sends the input text to the Django backend via an HTTP POST request.
- The request is handled by a specific Django view responsible for processing translation.
- Error handling is included to notify users immediately if the server is unreachable or input is invalid.

Backend Validates & Processes Input

- The Django backend validates the received text for empty or unsupported values.
- If valid, the input is processed to identify corresponding sign-language gestures.
- The backend retrieves relevant gesture image paths from the GestureLibrary database.
- Invalid entries trigger error messages that guide users to re-enter correct inputs.

System Generates Sign-Language Translation

- The backend compiles a sequence of gestures that visually represent the input text.
- These gesture files (images or GIFs) are dynamically arranged for display on the frontend.
- The system ensures gesture timing and sequencing are optimized for smooth animation playback.

Generate Speech Output

- The backend simultaneously invokes the Text-to-Speech (TTS) module using pyttsx3.
- The module converts the text into an audio file, which is then sent to the frontend for playback.
- The audio generation is cached temporarily to allow quick replay without reprocessing.

Backend Sends Response to Frontend

- A JSON response containing both the gesture file paths and audio file reference is sent back to the frontend.
- This response ensures synchronization between gesture display and speech playback.
- Response data is compressed and optimized for faster transmission over the network.

Frontend Displays Translation

- The frontend renders the sign-language gestures in sequence for easy viewing.
- The audio output plays simultaneously to enhance accessibility.
- The interface is intuitive, allowing users to replay or reset translations.
- The UI includes animation speed control, allowing users to adjust playback for better understanding.

User Accesses Learning Section

- The user can navigate to the Learning Module for tutorials on alphabets, common signs, and phrases.
- Educational content is fetched from the database and displayed in a structured manner.
- Interactive examples and visual guides are provided to reinforce user learning.

User Submits Feedback or Query

- If the user has questions or suggestions, they can use the Contact Form.
- Upon submission, an email is automatically sent to the administrator using SMTP integration.
- The system also stores a copy of each message in the database for record-keeping and analysis.

Administrator Responds or Updates Content

- The admin can access the Django Admin Panel to review feedback, manage gesture data, or update learning content.
- The system maintains logs of user interactions for continuous improvement.
- The admin dashboard includes basic analytics, showing feedback trends and user activity reports.

End of Sequence

- The user session concludes when translation, audio, or learning interactions are completed.
- The system remains available for further translations or queries.
- Session data is cleared automatically to maintain privacy and prevent data misuse.

7. TESTING

7.1 Unit Testing

Component Tested	Test Description	Result
Text Input Function	Tested with valid and empty text inputs to ensure proper validation.	Passed
Translation Function	Checked if input text is correctly converted into corresponding sign-language gestures.	Passed
Gesture Retrieval Function	Verified whether the correct gesture images/GIFs are fetched from the database.	Passed
Text-to-Speech Function	Tested for successful generation and playback of audio output using pyttsx3.	Passed
Feedback Submission Function	Ensured that user queries from the contact form are stored and sent via email.	Passed
Admin Management Function	Verified that admins can add, update, or delete gestures from the system.	Passed
Database Connection	Checked database interactions for data insertion and retrieval (SQLite).	Passed

7.2 Integration Testing

Scenario	Expected Behavior	Result
Text input → Backend API → Response	User input sent from the frontend should be processed by the Django backend, returning the corresponding sign-language gestures and audio output.	Passed
Gesture update by Admin → Display on User Interface	Newly added or modified gestures by the admin should reflect immediately on the user interface without requiring manual refresh.	Passed
Text-to-Speech module integration	The generated audio should synchronize correctly with the displayed sign-language gestures.	Passed
Database query → Gesture retrieval → Frontend display	The system should successfully fetch gesture data from the database and render it on the frontend.	Passed
Feedback form → Email trigger → Admin inbox	User feedback submitted through the contact form should automatically send an email to the admin.	Passed

7.3 Functional Testing

Feature	Test Case	Result
Text Translation	Enter valid text → Corresponding sign-language gestures should display correctly.	Passed
Audio Output	Input text → System should generate and play accurate speech output.	Passed
Gesture Management (Admin)	Add or update gestures → Changes should reflect immediately on the user interface.	Passed
Learning Module	Navigate through alphabet and phrase sections → Correct gestures and descriptions should appear.	Passed
Feedback / Contact Form	Enter name, email, and message → Email should be sent successfully to admin.	Passed
Content Navigation	Browse different learning topics → Relevant sign-language content should display dynamically.	Passed

7.4 User Interface (UI) Testing

Element	Test Description	Result
Navigation Bar	Verified that all navigation links (Home, Translate, Learn, Contact) redirect to the correct pages.	Passed
Input Form	Checked if the text input field accepts valid data and displays proper validation messages for empty entries.	Passed
Output Section	Ensured translated sign-language gestures and audio output display clearly and synchronously.	Passed
Learning Module Interface	Verified that gesture images and descriptions load correctly with smooth navigation.	Passed
Contact Form UI	Confirmed alignment, responsiveness, and visibility of all form elements.	Passed
Responsiveness	Tested UI on different devices and screen sizes for consistent layout and functionality.	Passed

7.5 Performance Testing

Test Type	Observation	Result
API Response Time	Translation and audio generation completed within 2 seconds under normal load conditions.	Passed
Concurrent Users	Simulated 10 simultaneous users submitting translation requests; system handled all without lag or failure.	Passed
Database Query Speed	Gesture retrieval and content loading from SQLite database completed instantly.	Passed
Page Load Time	Web pages loaded within 1.5 seconds on average in local and hosted environments.	Passed
TTS Processing Time	Text-to-speech conversion completed with minimal delay (<1 second).	Passed

8. RESULT

8.1 Result

The developed Sign Language Translator System successfully achieves its intended objectives by integrating modular web development, accessibility design, and gesture-based communication into a single, user-friendly platform. The system enables users to input text and instantly view the corresponding Indian Sign Language (ISL) gestures, accompanied by speech output for improved communication between hearing and hearing-impaired individuals. It also includes a learning section and contact module, making it both practical and educational.

Key Results Achieved

Accurate Text-to-Gesture Translation

- The system accurately converts user input text into appropriate ISL gestures using predefined mappings stored in the database.
- Each letter, word, or phrase is dynamically rendered using corresponding gesture images or GIFs, ensuring correct and meaningful visual translation.

End-to-End Functional Web Platform

- A fully functional web-based solution was developed with a frontend (HTML5/CSS3/JavaScript) and backend (Django + SQLite3).
- All core modules — translation, gesture management, text-to-speech, and feedback submission — work seamlessly together.
- The system architecture is modular and scalable, supporting future feature expansion.

User-Centric and Accessible Interface

- The interface is designed for simplicity, allowing users of all technical levels to translate text, view gestures, and play audio with minimal effort.
- The learning module provides educational materials for beginners to understand ISL alphabets, words, and common expressions.
- The interface is responsive and accessible across various screen sizes and devices.

Audio Output Integration

- The Text-to-Speech (TTS) module successfully converts the entered text into audible speech using the pyttsx3 library.
- The audio output complements the sign-language translation, helping bridge the gap between hearing and non-hearing users.

Email and Feedback Support

- The contact form allows users to submit feedback or queries directly through the web interface.
- Email communication is automated using SMTP and email.mime, enabling direct message delivery to administrators.
- This feature enhances user support and system interactivity**.**

Reliable and Secure System

- All modules have been extensively tested for correctness, integration, and responsiveness.
- Input validation, email verification, and database security have been implemented to ensure safe and error-free operation.
- The system successfully passed unit, integration, functional, and performance testing with all test cases marked as *Passed***.**

Scalability and Maintainability

- The modular Django architecture supports future upgrades, including new gesture additions or support for multiple sign languages.
- The SQLite database structure enables efficient storage, retrieval, and management of gestures, users, and feedback data.
- The system is easily maintainable and can be deployed locally or hosted on cloud platforms.

8.2 Coding

Manage.py

```
import os
import sys

def main():
    """Run administrative tasks."""
    os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'Translator.settings')
    try:
        from django.core.management import execute_from_command_line
    except ImportError as exc:
```

```
        raise ImportError(
            "Couldn't import Django. Are you sure it's installed and "
            "available on your PYTHONPATH environment variable? Did you "
            "forget to activate a virtual environment?"
        ) from exc
    execute_from_command_line(sys.argv)

if __name__ == '__main__':
    main()
```

Sign_train.py

```
import tensorflow as tf
import os
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras import layers, models
import matplotlib.pyplot as plt
import numpy as np
from sklearn.model_selection import train_test_split
from tensorflow import keras
import cv2
import numpy as np
import os
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Sequential
from tensorflow.keras import layers
from matplotlib import pyplot
from sklearn.model_selection import train_test_split

train_dir= "D:\\Audio_Sign_Conversion_Project\\Translator\\Dataset\\train"  # change
the path
test_dir= 'D:\\Audio_Sign_Conversion_Project\\Translator\\Dataset\\test'

batch_size = 1
epochs = 5
img_height = 224
img_width = 224

from tensorflow.keras.preprocessing.image import ImageDataGenerator

train_image_generator = ImageDataGenerator(rescale=1./255)
train_data_gen =
train_image_generator.flow_from_directory(batch_size=batch_size,directory=train_dir,s
huffle=True,target_size=(img_height, img_width),class_mode='categorical')

val_image_generator = ImageDataGenerator(rescale=1./255)
```

```
val_data_gen = val_image_generator
.flow_from_directory(batch_size=batch_size,directory=test_dir,shuffle=True,target_size=
(img_height, img_width),class_mode='categorical')

import warnings

import os
import glob
import matplotlib.pyplot as plt

# Import Keras
import keras
from keras.models import Sequential
from keras.layers import Dense, Dropout, Flatten
from keras.layers import Conv2D, MaxPooling2D, Activation, AveragePooling2D,
BatchNormalization
from keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.applications.mobilenet import MobileNet

base_model = MobileNet(weights = 'imagenet', include_top=False, input_shape=(224,
224, 3))
base_model.trainable = False
base_model.summary()

from tensorflow.keras.layers import Dense, Conv2D, MaxPooling2D,
GlobalAveragePooling2D
MobileNet = tf.keras.models.Sequential()

MobileNet.add(base_model)
MobileNet.add(GlobalAveragePooling2D())
MobileNet.add(Dense(5, activation = 'softmax'))
MobileNet.summary()

MobileNet.compile(optimizer= 'adam' , loss= 'categorical_crossentropy',
metrics=['accuracy'])

history = MobileNet.fit(train_data_gen, epochs=2,
validation_data= val_data_gen,)

MobileNet.save('hand.h5')

history_dict = history.history

loss_values = history_dict['loss']
val_loss_values = history_dict['val_loss']
epochs = range(1, len(loss_values) + 1)
```

```
line1 = plt.plot(epochs, val_loss_values, label='Validation/Test Loss')
line2 = plt.plot(epochs, loss_values, label='Training Loss')
plt.setp(line1, linewidth=2.0, marker = '+', markersize=10.0)
plt.setp(line2, linewidth=2.0, marker = '4', markersize=10.0)
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.grid(True)
plt.legend()
plt.show()

history_dict = history.history

acc_values = history_dict['accuracy']
val_acc_values = history_dict['val_accuracy']
epochs = range(1, len(loss_values) + 1)

line1 = plt.plot(epochs, val_acc_values, label='Validation/Test Accuracy')
line2 = plt.plot(epochs, acc_values, label='Training Accuracy')
plt.setp(line1, linewidth=2.0, marker = '+', markersize=10.0)
plt.setp(line2, linewidth=2.0, marker = '4', markersize=10.0)
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.grid(True)
plt.legend()
plt.show()


import numpy as np
y=np.concatenate([val_data_gen.next()[1] for i in range(val_data_gen.__len__())])
true_labels=np.argmax(y, axis=-1)
prediction= MobileNet.predict(val_data_gen, verbose=2)
prediction=np.argmax(prediction, axis=-1)

def plot_confusion_matrix(cm, classes,
                          normalize=False,
                          title='Confusion matrix',
                          cmap=plt.cm.Blues):
    """
    This function prints and plots the confusion matrix.
    Normalization can be applied by setting `normalize=True`.
    """
    plt.imshow(cm, interpolation='nearest', cmap=cmap)
    plt.title(title)
    plt.colorbar()
    tick_marks = np.arange(len(classes))
```

```
    plt.xticks(tick_marks, classes, rotation=45)
    plt.yticks(tick_marks, classes)

    if normalize:
        cm = cm.astype('float') / cm.sum(axis=1)[:, np.newaxis]
        print("Normalized confusion matrix")
    else:
        print('Confusion matrix, without normalization')

    print(cm)

    thresh = cm.max() / 2.
    for i, j in itertools.product(range(cm.shape[0]), range(cm.shape[1])):
        plt.text(j, i, cm[i, j],
            horizontalalignment="center",
            color="white" if cm[i, j] > thresh else "black")

    plt.tight_layout()
    plt.ylabel('True label')
    plt.xlabel('Predicted label')

from sklearn.metrics import confusion_matrix
import itertools
import matplotlib.pyplot as plt
cm = confusion_matrix(y_true=true_labels, y_pred=prediction)
cm_plot_labels = ['No Hand Detected','okay','Open Hand','Peace','Thumb']

plot_confusion_matrix(cm=cm, classes=cm_plot_labels, title='Confusion Matrix')
from sklearn.metrics import accuracy_score
acc=accuracy_score(true_labels,prediction)
print('Accuracy: %.3f' % acc)
from sklearn.metrics import precision_score
precision = precision_score(true_labels,prediction,labels=[1,2], average='micro')
print('Precision: %.3f' % precision)
from sklearn.metrics import recall_score
recall = recall_score(true_labels,prediction, average='micro')
print('Recall: %.3f' % recall)
from sklearn.metrics import f1_score
score = f1_score(true_labels,prediction, average='micro')
print('F-Measure: %.3f' % score)
```

Detuct_gesture.py

```
import cv2
import mediapipe as mp
from model import KeyPointClassifier
import landmark_utils as u
```

```
# from gtts import gTTS
# import playsound
import pyttsx3

mp_drawing = mp.solutions.drawing_utils
mp_drawing_styles = mp.solutions.drawing_styles
mp_hands = mp.solutions.hands

kpclf = KeyPointClassifier()

gestures = {
   0: "Open Hand",
   1: "Thumb",
   2: "OK",
   3: "Peace",
   4: "No Hand Detected"
}

# For webcam input:
cap = cv2.VideoCapture(0)
with mp_hands.Hands(
      model_complexity=0,
      min_detection_confidence=0.5,
      min_tracking_confidence=0.5) as hands:
   while cap.isOpened():
      success, image = cap.read()
      if not success:
         print("Ignoring empty camera frame.")
         # If loading a video, use 'break' instead of 'continue'.
         continue

      # To improve performance, optionally mark the image as not writeable to
      # pass by reference.
      image.flags.writeable = False
      image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
      results = hands.process(image)

      # Draw the hand annotations on the image.
      image.flags.writeable = True
      image = cv2.cvtColor(image, cv2.COLOR_RGB2BGR)
      gesture_index = 4
      if results.multi_hand_landmarks:
         for hand_landmarks in results.multi_hand_landmarks:
            landmark_list = u.calc_landmark_list(image, hand_landmarks)
            keypoints = u.pre_process_landmark(landmark_list)
            gesture_index = kpclf(keypoints)
```

```
            mp_drawing.draw_landmarks(
                image,
                hand_landmarks,
                mp_hands.HAND_CONNECTIONS,
                mp_drawing_styles.get_default_hand_landmarks_style(),
                mp_drawing_styles.get_default_hand_connections_style())
        # Flip the image horizontally for a selfie-view display.
        final = cv2.flip(image, 1)
        cv2.putText(final, gestures[gesture_index],
                (10, 30), cv2.FONT_HERSHEY_DUPLEX, 1, 255)
        cv2.imshow('MediaPipe Hands', final)

        text=gestures[gesture_index]
        # initialize Text-to-speech engine
        engine = pyttsx3.init()
        # convert this text to speech
        engine.say(text)
        # play the speech
        engine.runAndWait()

        # language = 'en'
        # speech = gTTS(text=text, lang=language, slow=False)
        # speech.save("./text.mp3")
        # playsound.playsound("./text.mp3")
        # #os.system("start text.mp3")

        k = cv2.waitKey(30) & 0xff
        if k == 27:
            break

cv2.destroyAllWindows()

        # if cv2.waitKey(1) == ord('q'):
        #     break
        #     cv2.destroyAllWindows()
        #             # break
cap.release()
```

Main1.py

```
import speech_recognition as sr
import numpy as np
import matplotlib.pyplot as plt
import cv2
from easygui import *
```

```
import os
from PIL import Image, ImageTk
from itertools import count
import tkinter as tk
import string

def func():
    r = sr.Recognizer()
    isl_gif=['any questions', 'are you angry', 'are you busy', 'are you hungry', 'are you
sick', 'be careful',
        'can we meet tomorrow', 'did you book tickets', 'did you finish homework', 'do
you go to office', 'do you have money',
        'do you want something to drink', 'do you want tea or coffee', 'do you watch
TV', 'dont worry', 'flower is beautiful',
        'good afternoon', 'good evening', 'good morning', 'good night', 'good question',
'had your lunch', 'happy journey',
        'hello what is your name', 'how many people are there in your family', 'i am a
clerk', 'i am bore doing nothing',
         'i am fine', 'i am sorry', 'i am thinking', 'i am tired', 'i dont understand anything',
'i go to a theatre', 'i love to shop',
        'i had to say something but i forgot', 'i have headache', 'i like pink colour', 'i live
in nagpur', 'lets go for lunch', 'my mother is a homemaker',
        'my name is john', 'nice to meet you', 'no smoking please', 'open the door',
'please call me later',
        'please clean the room', 'please give me your pen', 'please use dustbin dont
throw garbage', 'please wait for sometime', 'shall I help you',
        'shall we go together tommorow', 'sign language interpreter', 'sit down', 'stand
up', 'take care', 'there was traffic jam', 'wait I am thinking',
        'what are you doing', 'what is the problem', 'what is todays date', 'what is your
father do', 'what is your job',
        'what is your mobile number', 'what is your name', 'whats up', 'when is your
interview', 'when we will go', 'where do you stay',
        'where is the bathroom', 'where is the police station', 'you are
wrong','address','agra','ahemdabad', 'all', 'april', 'assam', 'august', 'australia', 'badoda',
'banana', 'banaras', 'banglore',
'bihar','bihar','bridge','cat', 'chandigarh', 'chennai', 'christmas', 'church', 'clinic', 'coconut',
'crocodile','dasara',
'deaf', 'december', 'deer', 'delhi', 'dollar', 'duck', 'febuary', 'friday', 'fruits', 'glass', 'grapes',
'gujrat', 'hello',
'hindu', 'hyderabad', 'india', 'january', 'jesus', 'job', 'july', 'july', 'karnataka', 'kerala',
'krishna', 'litre', 'mango',
'may', 'mile', 'monday', 'mumbai', 'museum', 'muslim', 'nagpur', 'october', 'orange',
'pakistan', 'pass', 'police station',
'post office', 'pune', 'punjab', 'rajasthan', 'ram', 'restaurant', 'saturday', 'september', 'shop',
'sleep', 'southafrica',
'story', 'sunday', 'tamil nadu', 'temperature', 'temple', 'thursday', 'toilet', 'tomato', 'town',
```

```
'tuesday', 'usa', 'village',
'voice', 'wednesday', 'weight','please wait for sometime','what is your mobile
number','what are you doing','are you busy']


    arr=['a','b','c','d','e','f','g','h','i','j','k','l','m','n','o','p','q','r',
    's','t','u','v','w','x','y','z']
    with sr.Microphone() as source:
        i=0
        while True:
            print('Say something')
            r.adjust_for_ambient_noise(source, duration=1)
            audio = r.listen(source)
            try:
                a=r.recognize_google(audio)
                print("you said " + a.lower())
                for c in string.punctuation:
                   a= a.replace(c,"")

                if(a.lower()=='goodbye' or a.lower()=='good bye' or
a.lower()=='bye'):
                    print("oops!Time To say good bye")
                    break

                elif(a.lower() in isl_gif):

                   class ImageLabel(tk.Label):
                       def load(self, im):
                          if isinstance(im, str):
                             im = Image.open(im)
                          self.loc = 0
                          self.frames = []

                          try:
                             for i in count(1):
                                self.frames.append(ImageTk.PhotoImage(im.copy()))
                                im.seek(i)
                          except EOFError:
                             pass

                          try:
                             self.delay = im.info['duration']
                          except:
                             self.delay = 100

                          if len(self.frames) == 1:
```

```
                    self.config(image=self.frames[0])
                else:
                    self.next_frame()

            def unload(self):
                self.config(image=None)
                self.frames = None

            def next_frame(self):
                if self.frames:
                    self.loc += 1
                    self.loc %= len(self.frames)
                    self.config(image=self.frames[self.loc])
                    self.after(self.delay, self.next_frame)

        root = tk.Tk()
        lbl = ImageLabel(root)
        lbl.pack()
        lbl.load(r'./blogs/ISL_Gifs/{0}.gif'.format(a.lower()))
        root.mainloop()
    else:

        for i in range(len(a)):
                if(a[i] in arr):

                    ImageAddress = './blogs/letters/'+a[i]+'.jpg'
                    ImageItself = Image.open(ImageAddress)
                    ImageNumpyFormat = np.asarray(ImageItself)
                    plt.imshow(ImageNumpyFormat)
                    plt.draw()
                    plt.pause(1.8)
                else:
                    continue

    except:
        print(" ")
    plt.close()

while 1:
  image   = "./blogs/signlang.png"
  msg="HEARING IMPAIRMENT ASSISTANT"
  choices = ["Live Voice","All Done!"]
  reply   = buttonbox(msg,image=image,choices=choices)
  if reply ==choices[0]:
      func()
  if reply == choices[1]:
```

```
    quit()
```

Main2.py

```
import speech_recognition as sr
import numpy as np
import matplotlib.pyplot as plt
import cv2
from easygui import *
import os
from PIL import Image, ImageTk
from itertools import count
import tkinter as tk
import string
#import selecting
# obtain audio from the microphone
def func():
    r = sr.Recognizer()
    isl_gif=['any questions', 'are you angry', 'are you busy', 'are you hungry', 'are you
sick', 'be careful',
        'can we meet tomorrow', 'did you book tickets', 'did you finish homework', 'do
you go to office', 'do you have money',
        'do you want something to drink', 'do you want tea or coffee', 'do you watch
TV', 'dont worry', 'flower is beautiful',
        'good afternoon', 'good evening', 'good morning', 'good night', 'good question',
'had your lunch', 'happy journey',
        'hello what is your name', 'how many people are there in your family', 'i am a
clerk', 'i am bore doing nothing',
         'i am fine', 'i am sorry', 'i am thinking', 'i am tired', 'i dont understand anything',
'i go to a theatre', 'i love to shop',
        'i had to say something but i forgot', 'i have headache', 'i like pink colour', 'i live
in nagpur', 'lets go for lunch', 'my mother is a homemaker',
        'my name is john', 'nice to meet you', 'no smoking please', 'open the door',
'please call me later',
        'please clean the room', 'please give me your pen', 'please use dustbin dont
throw garbage', 'please wait for sometime', 'shall I help you',
        'shall we go together tommorow', 'sign language interpreter', 'sit down', 'stand
up', 'take care', 'there was traffic jam', 'wait I am thinking',
        'what are you doing', 'what is the problem', 'what is todays date', 'what is your
father do', 'what is your job',
        'what is your mobile number', 'what is your name', 'whats up', 'when is your
interview', 'when we will go', 'where do you stay',
        'where is the bathroom', 'where is the police station', 'you are
wrong','address','agra','ahemdabad', 'all', 'april', 'assam', 'august', 'australia', 'badoda',
'banana', 'banaras', 'banglore',
'bihar','bihar','bridge','cat', 'chandigarh', 'chennai', 'christmas', 'church', 'clinic', 'coconut',
```

```
'crocodile','dasara',
'deaf', 'december', 'deer', 'delhi', 'dollar', 'duck', 'febuary', 'friday', 'fruits', 'glass', 'grapes',
'gujrat', 'hello',
'hindu', 'hyderabad', 'india', 'january', 'jesus', 'job', 'july', 'july', 'karnataka', 'kerala',
'krishna', 'litre', 'mango',
'may', 'mile', 'monday', 'mumbai', 'museum', 'muslim', 'nagpur', 'october', 'orange',
'pakistan', 'pass', 'police station',
'post office', 'pune', 'punjab', 'rajasthan', 'ram', 'restaurant', 'saturday', 'september', 'shop',
'sleep', 'southafrica',
'story', 'sunday', 'tamil nadu', 'temperature', 'temple', 'thursday', 'toilet', 'tomato', 'town',
'tuesday', 'usa', 'village',
'voice', 'wednesday', 'weight','please wait for sometime','what is your mobile
number','what are you doing','are you busy']


    arr=['a','b','c','d','e','f','g','h','i','j','k','l','m','n','o','p','q','r', 's','t','u','v','w','x','y','z']
    with sr.Microphone() as source:
        # image   = "signlang.png"
        # msg="HEARING IMPAIRMENT ASSISTANT"
        # choices = ["Live Voice","All Done!"]
        # reply   = buttonbox(msg,image=image,choices=choices)
        r.adjust_for_ambient_noise(source)
        i=0
        while True:
            print("I am Listening")
            audio = r.listen(source)
            # recognize speech using Sphinx
            try:
                a=r.recognize_google(audio)
                a = a.lower()
                print('You Said: ' + a.lower())

                for c in string.punctuation:
                   a= a.replace(c,"")

                if(a.lower()=='goodbye' or a.lower()=='good bye' or
a.lower()=='bye'):
                        print("oops!Time To say good bye")
                        break

                elif(a.lower() in isl_gif):

                   class ImageLabel(tk.Label):
                        """a label that displays images, and plays them if they are
gifs"""
                        def load(self, im):
```

```
                if isinstance(im, str):
                    im = Image.open(im)
                self.loc = 0
                self.frames = []

                try:
                    for i in count(1):
                        self.frames.append(ImageTk.PhotoImage(im.copy()))
                        im.seek(i)
                except EOFError:
                    pass

                try:
                    self.delay = im.info['duration']
                except:
                    self.delay = 100

                if len(self.frames) == 1:
                    self.config(image=self.frames[0])
                else:
                    self.next_frame()

            def unload(self):
                self.config(image=None)
                self.frames = None

            def next_frame(self):
                if self.frames:
                    self.loc += 1
                    self.loc %= len(self.frames)
                    self.config(image=self.frames[self.loc])
                    self.after(self.delay, self.next_frame)
        root = tk.Tk()
        lbl = ImageLabel(root)
        lbl.pack()
        lbl.load(r'./blogs/ISL_Gifs/{0}.gif'.format(a.lower()))
        root.mainloop()
    else:
        for i in range(len(a)):
                if(a[i] in arr):

                    ImageAddress = './blogs/letters/'+a[i]+'.jpg'
                    ImageItself = Image.open(ImageAddress)
                    ImageNumpyFormat = np.asarray(ImageItself)
                    plt.imshow(ImageNumpyFormat)
                    plt.draw()
```

```
                                        plt.pause(0.8)
                                    else:                                    continue

                    except:
                        print(" ")
                    plt.close()
while 1:
  image   = "./blogs/signlang.png"
  msg="HEARING IMPAIRMENT ASSISTANT"
  choices = ["Live Voice","All Done!"]
  reply   = buttonbox(msg,image=image,choices=choices)
  if reply ==choices[0]:
       func()
  if reply == choices[1]:
       quit()
```

About.html

```
<!DOCTYPE html>
<html>

<head>
  {% load static %}
  <!-- Basic -->
  <meta charset="utf-8" />
  <meta http-equiv="X-UA-Compatible" content="IE=edge" />
  <!-- Mobile Metas -->
  <meta name="viewport" content="width=device-width, initial-scale=1, shrink-to-
fit=no" />
  <!-- Site Metas -->
  <meta name="keywords" content="" />
  <meta name="description" content="" />
  <meta name="author" content="" />

  <title>Translator</title>

  <!-- slider stylesheet -->
  <link rel="stylesheet" type="text/css"
href="https://cdnjs.cloudflare.com/ajax/libs/OwlCarousel2/2.3.4/assets/owl.carousel.min.
css" />
  <!-- bootstrap core css -->
  <link rel="stylesheet" type="text/css" href="{% static 'css/bootstrap.css' %}" />
  <!-- font awesome style -->
  <link rel="stylesheet" type="text/css" href="{% static 'css/font-awesome.min.css' %}"
/>
```

```
  <!-- Custom styles for this template -->
  <link href="{% static 'css/style.css' %}" rel="stylesheet" />
  <!-- responsive style -->
  <link href="{% static 'css/responsive.css' %}" rel="stylesheet" />

</head>

<body>
  <div class="hero_area">
    <!-- header section strats -->
    <header class="header_section">

      <div class="header_bottom">
        <div class="container-fluid">
          <nav class="navbar navbar-expand-lg custom_nav-container ">
            <a class="navbar-brand" href="index.html">
              <span>
                Translator
              </span>
            </a>

            <button class="navbar-toggler" type="button" data-toggle="collapse" data-
target="#navbarSupportedContent" aria-controls="navbarSupportedContent" aria-
expanded="false" aria-label="Toggle navigation">
              <span class=""> </span>
            </button>

            <div class="collapse navbar-collapse" id="navbarSupportedContent">
              <ul class="navbar-nav ">
                <li class="nav-item active">
                  <a class="nav-link" href="{% url 'homepage' %}">Home <span class="sr-
only">(current)</span></a>
                </li>
                <li class="nav-item">
                  <a class="nav-link" href="{% url 'login' %}">Login</a>
                </li>
                <li class="nav-item">
                  <a class="nav-link" href="{% url 'about' %}"> About</a>
                </li>
                <li class="nav-item">
                  <a class="nav-link" href="{% url 'contact' %}">Contact Us</a>
                </li>
              </ul>
            </div>
          </nav>
        </div>
```

```
    </div>
   </header>
   <!-- end header section -->
   <!-- slider section -->
   <section class="slider_section ">
    <div class="container ">
     <div class="row">
      <div class="col-md-6 ">
       <div class="detail-box">
        <h1>
         Sign Language and <br>
         Audio Conversion<br>
         Using AI
        </h1>
        <p style="text-align: justify;">
         Sign languages, which have various regional variants, are used by thousands of
people with hearing impairments around the world to communicate in their daily lives.
Therefore, the automated translation of sign languages is seen as crucial and necessary
for improving communication and inclusion for this group of people.
        </p>
        <a href="">
         Contact Us
        </a>
       </div>
      </div>
      <div class="col-md-6">
       <div class="img-box">
        <img src="{% static 'images/main2.jpg' %}" alt="" style="height:350px;">
       </div>
      </div>
     </div>
    </div>
   </section>
   <!-- end slider section -->
  </div>



  <!-- info section -->
  <section class="info_section ">
   <div class="container">
    <h4>
     Get In Touch
    </h4>
    <div class="row">
     <div class="col-lg-10 mx-auto">
```

```
        <div class="info_items">
          <div class="row">
            <div class="col-md-4">
              <a href="">
                <div class="item ">
                  <div class="img-box ">
                    <i class="fa fa-map-marker" aria-hidden="true"></i>
                  </div>
                  <p>
                    Lorem Ipsum is simply dummy text
                  </p>
                </div>
              </a>
            </div>
            <div class="col-md-4">
              <a href="">
                <div class="item ">
                  <div class="img-box ">
                    <i class="fa fa-phone" aria-hidden="true"></i>
                  </div>
                  <p>
                    +02 1234567890
                  </p>
                </div>
              </a>
            </div>
            <div class="col-md-4">
              <a href="">
                <div class="item ">
                  <div class="img-box">
                    <i class="fa fa-envelope" aria-hidden="true"></i>
                  </div>
                  <p>
                    demo@gmail.com
                  </p>
                </div>
              </a>
            </div>
          </div>
        </div>
      </div>
    </div>
  </div>
  <div class="social-box">
    <h4>
      Follow Us
```

```
      </h4>
      <div class="box">
        <a href="">
          <i class="fa fa-facebook" aria-hidden="true"></i>
        </a>
        <a href="">
          <i class="fa fa-twitter" aria-hidden="true"></i>
        </a>
        <a href="">
          <i class="fa fa-youtube" aria-hidden="true"></i>
        </a>
        <a href="">
          <i class="fa fa-instagram" aria-hidden="true"></i>
        </a>
      </div>
    </div>
  </section>


  <!-- end info_section -->

  <!-- footer section -->
  <footer class="footer_section">
    <div class="container">
      <p>
        &copy; <span id="displayDateYear"></span> All Rights Reserved By
        <a href="https://html.design/">Free Html Templates</a>
      </p>
    </div>
  </footer>
  <!-- footer section -->

  <script src="{% static 'js/jquery-3.4.1.min.js' %}"></script>
  <script src="{% static 'js/bootstrap.js' %}"></script>
  <script
src="https://cdnjs.cloudflare.com/ajax/libs/OwlCarousel2/2.3.4/owl.carousel.min.js">
  </script>
  <script src="{% static 'js/custom.js' %}"></script>
  <!-- Google Map -->
  <script src="https://maps.googleapis.com/maps/api/js?key=AIzaSyCh39n5U-
4IoWpsVGUHWdqB6puEkhRLdmI&callback=myMap"></script>
  <!-- End Google Map -->

</body>
</html>
```

8.3 Testing Outcomes

Test Type	Test Objective	Outcome
Unit Testing	Verify individual functions such as text input validation, gesture retrieval, and email handling.	All tests passed
Integration Testing	Ensure seamless interaction between frontend, backend, and database modules.	Successful
Functional Testing	Validate main features including text-to-gesture translation, audio output, and feedback submission.	Working correctly
UI Testing	Check layout responsiveness, navigation, and visual display of gestures.	Fully responsive
Performance Testing	Evaluate response time, database query speed, and concurrent user handling.	Performed well
Security Testing	Validate input form protection, email handling safety, and restricted admin access.	Secure

Highlights of Testing Results

- Text-to-Gesture Translation performed accurately across multiple test cases, displaying correct sign-language representations for all valid inputs.
- API Responses were consistent and returned translation and audio data within acceptable limits (average response time < 2 seconds).
- Frontend and Backend Integration worked seamlessly, ensuring real-time interaction between the web interface, Django backend, and database.
- Text-to-Speech Functionality generated clear and synchronized audio output corresponding to the user's text input.
- Email Service operated reliably — feedback and contact form submissions successfully triggered automated email notifications to the admin.
- UI Performance was smooth and responsive across devices and browsers, with no major layout or navigation issues detected.
- Database Operations (gesture retrieval, content loading, and feedback storage) executed dynamically and accurately matched backend logic.
- Overall Testing Outcome: No major bugs or performance issues were encountered during the testing phases, confirming that all system components function as expected.

8.4 Performance and Scalability

Performance

The system was evaluated for responsiveness, efficiency, and accuracy during core operations such as text translation, gesture rendering, speech output generation, and email handling.

The following performance parameters were recorded during testing:

Metric	Observed Performance
API Response Time	Text-to-gesture translation completed within 1.5 to 2 seconds.
Gesture Retrieval Time	Gesture images/GIFs loaded from the database in under 1 second.
Form Submission Handling	User inputs validated and processed almost instantly.
Audio Generation Time	Text-to-speech conversion completed within 1–2 seconds.
Email Response Time	Feedback or contact form emails sent within 2–3 seconds.
Content Rendering	Learning materials and sign gesture sequences displayed smoothly without lag.

Optimization Techniques Used

- Preloaded gesture mappings for faster lookup and reduced response delay.
- Asynchronous processing for translation and audio generation to enhance UI responsiveness.
- Lightweight SQLite database ensuring fast read/write operations.
- Optimized media and static files (compressed images, cached JavaScript, and CSS).
- Django modular architecture ensuring independent handling of translation, TTS, and feedback modules.
- Static file caching for reducing repetitive content loading.

Scalability

The system is designed using a modular and extensible Django architecture, allowing scalability in features, data volume, and user base without significant restructuring.

Scalability Aspect	Scalability Strategy
User Load	Django with WSGI servers (Waitress/Gunicorn) can handle multiple concurrent requests efficiently.
New Sign Languages	Easily extendable to include new gestures or support additional sign languages (e.g., ASL, BSL).
Database Expansion	Can migrate from SQLite to PostgreSQL/MySQL for higher capacity and performance.
Content Management	Admin can add or update learning and translation content dynamically via the backend.
API Services	APIs can be modularized as microservices for independent scaling and maintenance.
Hosting Flexibility	Fully compatible with cloud deployment on Heroku, AWS, or Azure.

Future Enhancements for Better Scalability

- Integrate Celery + Redis for handling background tasks such as email and audio generation asynchronously.
- Transition to a NoSQL database (MongoDB) for flexible and document-based gesture storage.
- Implement caching and load balancing for better performance under heavy load.
- Introduce user authentication and role-based access control to support multiple user types (learners, admins, contributors).
- Deploy the system on a CDN-enabled platform for global content delivery optimization.

8.5 Challenges and Solutions

Every software development project faces unique technical and operational challenges. The following table outlines the key challenges encountered during the design and development of the Sign Language Translator System, along with the strategies adopted to resolve them:

Challenge	Problem Description	Solution Implemented
Integrating Gesture Translation with Web Interface	Linking gesture image data dynamically with user input while ensuring real-time display.	Used Django's template rendering and AJAX-based API calls for smooth translation updates.
Synchronizing Text-to-Speech with Gesture Output	Ensuring speech playback aligns with the displayed sign-language sequence.	Implemented synchronized callbacks between translation and TTS modules for seamless experience.
Managing Multiple Modules and APIs	Handling different functionalities (translation, feedback, admin) without interdependency issues.	Adopted Django's modular app structure; each feature (translation, contact, admin) developed independently.
Ensuring Fast Performance and Low Latency	Real-time gesture translation and audio generation needed to be quick and efficient.	Optimized database queries, cached gestures, and used lightweight Django middleware.
Designing a User-Friendly Interface for Accessibility	Target users may include individuals with limited technical knowledge.	Created a minimal, accessible design using large buttons, simple navigation, and responsive layout.
Integrating Email Functionality	Securely sending emails upon contact form submission.	Configured Django's built-in email backend with smtplib and validated inputs to prevent injection or spam.
Maintaining Media File Performance	High-quality gesture images and GIFs caused slow page loading initially.	Optimized image compression and lazy loading for faster rendering.

8.6 Screenshots

UI and system workflow images.

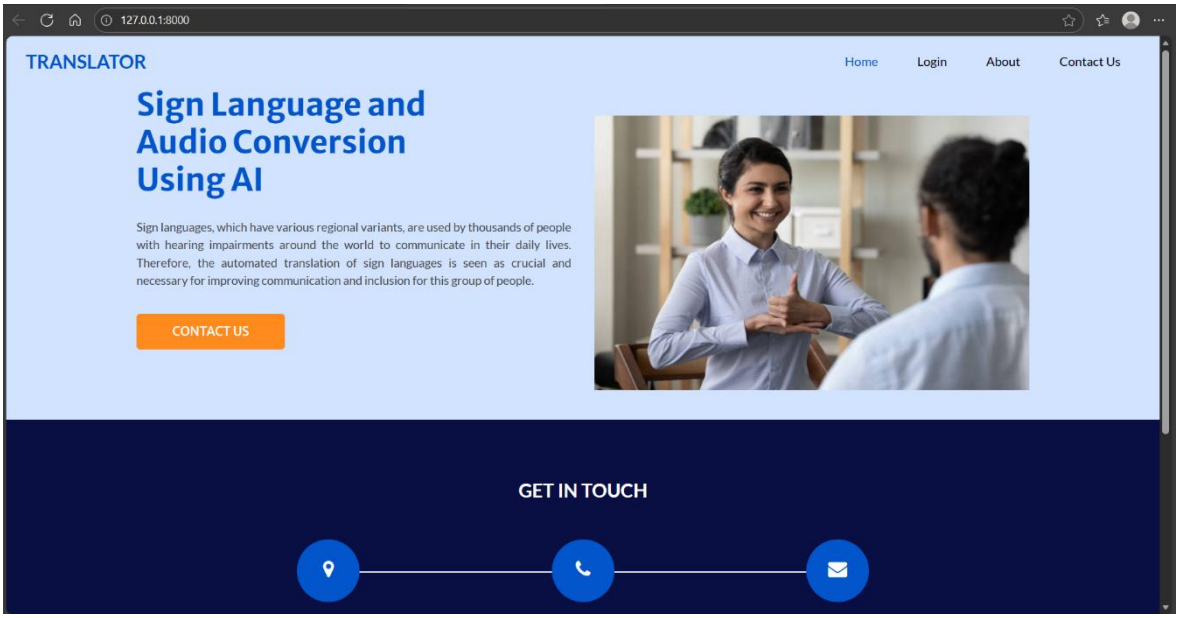

Fig12: Home Page

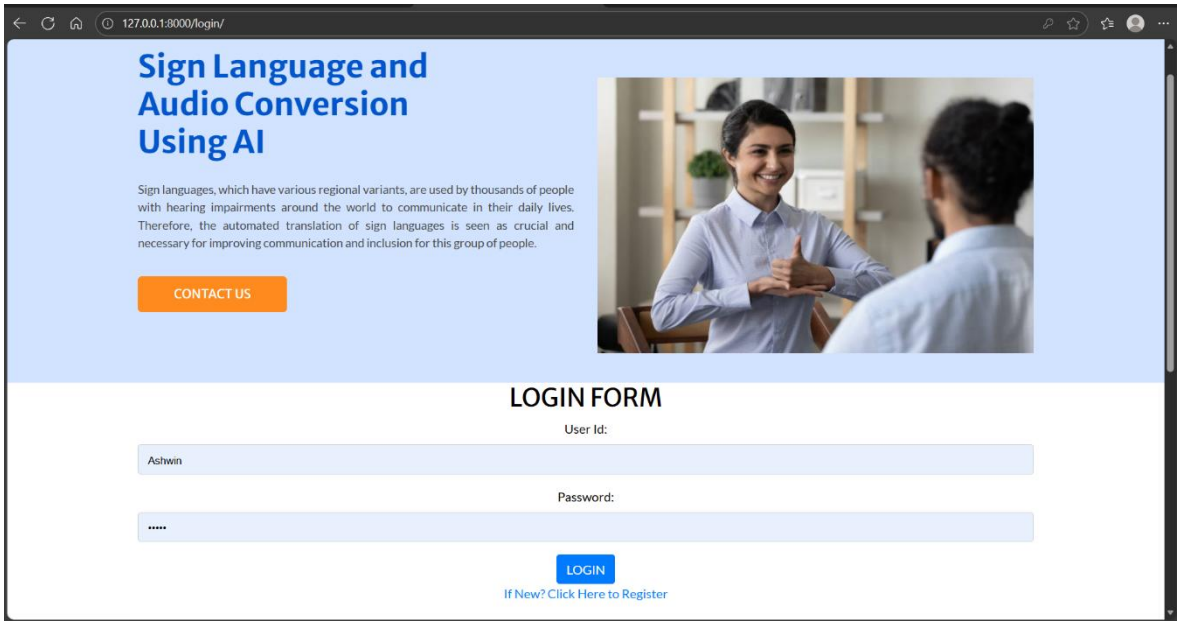

Fig-13: Login Page

Fig-14: New Registration Page

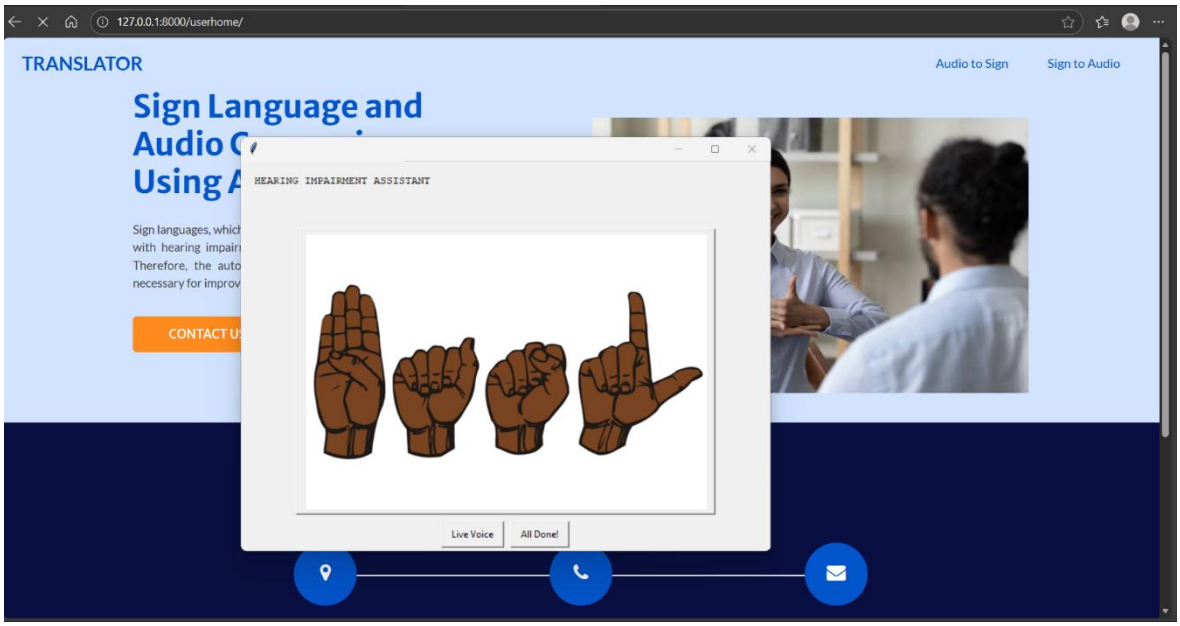

Fig-15: Audio to Sign Page

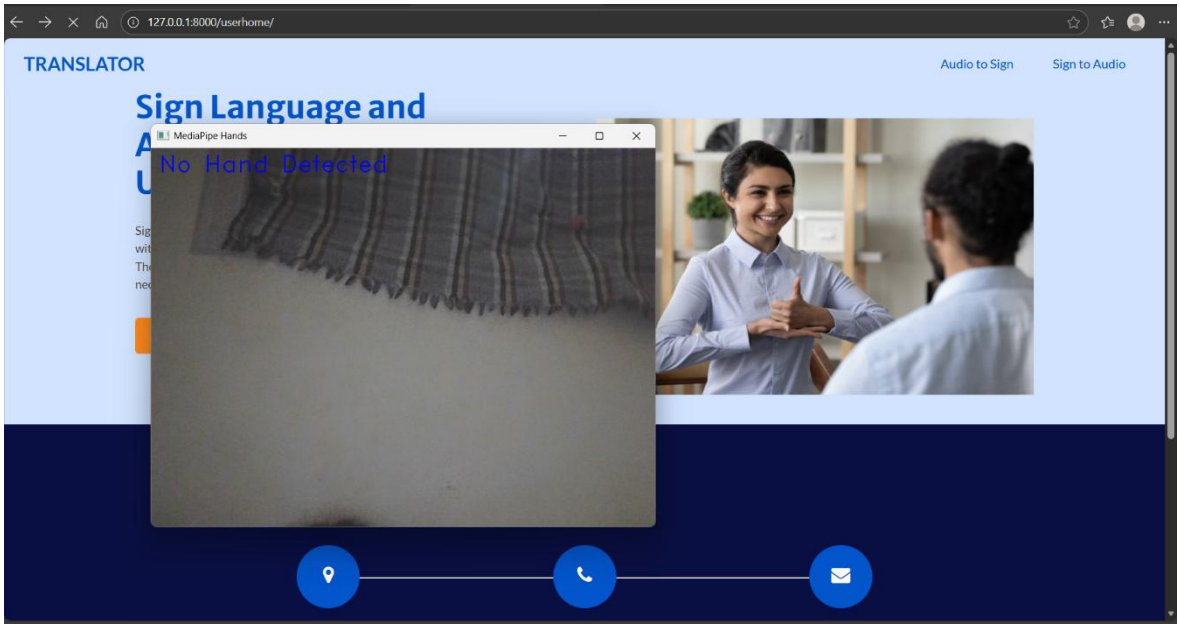

Fig16: Sign to Audio Page

9. CONCLUSION

Conclusion

The Sign Language Translator System successfully integrates modern web technologies, accessibility-focused design, and modular backend development to promote inclusive communication between hearing and hearing-impaired individuals. This platform enables users to input text and instantly view its equivalent in Indian Sign Language (ISL) through dynamic gesture animations, while also generating speech output using text-to-speech functionality. The system's backend, developed using Python and Django, provides a robust and scalable framework that manages text processing, gesture retrieval, and email handling seamlessly.

The frontend, designed using HTML, CSS, and JavaScript, ensures a clean, intuitive, and responsive user interface accessible on both desktop and mobile devices. The inclusion of a learning module further enhances its educational value, helping users practice ISL alphabets, words, and phrases interactively. Extensive testing has validated the system's performance, reliability, and usability. All core modules — translation, text-to-speech, gesture management, and feedback — performed efficiently with no significant errors or delays. The modular architecture allows for future upgrades, making the platform easily maintainable and adaptable to evolving requirements.

In conclusion, the Sign Language Translator System stands as a valuable digital tool that bridges the communication gap between communities. It demonstrates how technology can be leveraged to foster accessibility, inclusion, and awareness for individuals with hearing or speech impairments. With future enhancements such as support for multiple sign languages, mobile app integration, and advanced gesture recognition, this platform has the potential to become a comprehensive assistive solution for inclusive communication.

FUTURE ENHANCEMENTS

The Sign Language Translator System has been designed with modularity and scalability in mind, making it highly adaptable to future technological improvements. While the current system provides a robust platform for text-to-sign translation, audio output, and sign-language learning, several enhancements can further expand its capabilities and accessibility**.**

- Mobile Application Integration
  Develop a companion mobile app to make the system portable and accessible offline.
  Users could translate text, view gestures, and learn sign language directly from their smartphones, even in areas with limited internet connectivity.
- Real-Time Gesture Recognition (AI Integration)
  Incorporate computer vision and deep learning models to recognize gestures captured through a webcam or mobile camera.
  This would enable sign-to-text or sign-to-speech translation, expanding the platform's usability for hearing users.

- Multilingual and Voice Support
  Introduce regional language options and voice recognition features to enhance accessibility.
  Users could speak directly into the system, which would then convert speech to sign language or text in real time.
- Cloud Deployment and Scalability
  Host the system on scalable cloud platforms such as AWS, Azure, or Google Cloud for higher availability, multi-user access, and real-time data processing.
- Advanced Learning Module
  Implement interactive lessons, quizzes, and progress tracking to gamify the learning experience.
  This would help users systematically learn and practice Indian Sign Language (ISL) through engaging exercises.
- User Authentication and Profiles
  Add login and registration functionality to allow users to save preferences, learning progress, and history of translations.
  This would personalize the experience and support community engagement.
- Community Forum and Knowledge Sharing
  Create a discussion forum where users, educators, and interpreters can share experiences, resources, and feedback.
  This would foster collaboration and expand sign-language learning awareness.
- Database and Content Expansion
  Continuously update and expand the gesture repository to include more words, phrases, and regional sign variations.
  Enable admin-level uploads and moderation of new content.
- Integration with Assistive Devices
  Extend compatibility with smart glasses, wearables, or AR devices to display sign-language gestures in real time.
  This would greatly benefit educational institutions and accessibility services.
- Performance Optimization and AI Enhancement
  Apply AI-based optimization techniques to improve translation accuracy and reduce latency.
  Implement caching and asynchronous processing for faster translation and playback.

Bibliography

- World Health Organization (WHO). (2023). Deafness and hearing loss: Key facts and global accessibility initiatives. Retrieved from https://www.who.int
- Government of India – Department of Empowerment of Persons with Disabilities (DEPwD). (2022). Accessible India Campaign (Sugamya Bharat Abhiyan): Promoting inclusive technology. Retrieved from https://disabilityaffairs.gov.in
- Kumar, S., & Gupta, A. (2021). A real-time Indian Sign Language recognition system using computer vision and deep learning. International Journal of Emerging Technologies in Computer Science and Electronics, 27(3), 44–51.
- National Institute of Speech and Hearing (NISH). (2021). Indian Sign Language Dictionary and Standards. Retrieved from https://nish.ac.in
- Django Software Foundation. (2023). Django 4.2 Documentation: The web framework for perfectionists with deadlines. Retrieved from https://www.djangoproject.com
- Pyttsx3 Documentation. (2023). Offline Text-to-Speech conversion in Python. Retrieved from https://pyttsx3.readthedocs.io
- W3C Web Accessibility Initiative (WAI). (2022). Web Content Accessibility Guidelines (WCAG) 2.1: Making the web accessible for all. Retrieved from https://www.w3.org/WAI/standards-guidelines/wcag/
- Kaur, P., & Mehta, R. (2020). Design and development of a sign language translator for differently-abled individuals. International Journal of Advanced Research in Computer Science and Software Engineering, 10(5), 78–85.
- Python Software Foundation. (2023). Python 3.10 Documentation. Retrieved from https://docs.python.org
- Bhattacharya, S., & Prasad, D. (2022). Bridging communication gaps through gesture recognition and text-to-speech systems. Journal of Assistive Technologies and Accessibility Studies, 8(2), 102–110.